Quantum Computing for Quantum Chemistry

Student Number: 1822054

Candidate Number: 1101066

March 2026

This laboratory follows the Quantum Computing for Quantum Chemistry manual [1].

Import libraries

In [1]:
```
import numpy as np
from pyscf import gto
from openfermion import FermionOperator as FO
from pyscf import scf, fci
from IPython.display import Image
from openfermionpyscf import run_pyscf
from openfermion import MolecularData, get_fermion_operator
from openfermion import QubitOperator as ofQubitOperator
from openfermion.transforms import jordan_wigner
from qiskit_ibm_runtime import QiskitRuntimeService
from qiskit import QuantumCircuit
from qiskit.transpiler.preset_passmanagers import generate_preset_pass_manager
from qiskit.quantum_info import Statevector
from qiskit.visualization import plot_histogram
from qiskit_ibm_runtime import SamplerV2 as Sampler
from qiskit_aer import AerSimulator
import matplotlib.pyplot as plt
from qiskit import QuantumRegister, ClassicalRegister
from qiskit.circuit.library import QFTGate
from qiskit import transpile
import time
```

Part I

Conventional quantum chemistry

First-quantised Hamiltonians and wavefunctions

Under the Born-Oppenheimer approximation (fixed nuclei), the electronic Hamiltonian for N electrons and N_n nuclei is

$$\hat{H} = \hat{h} + \hat{g} + v_{nn}.$$

The one-electron (core) term is

$$\hat{h} = \sum_{i=1}^{N} \left(-\frac{1}{2}\nabla_i^2 - \sum_{A=1}^{N_n} \frac{Z_A}{|\mathbf{r}_i - \mathbf{R}_A|} \right),$$

with $\mathbf{r}_i$ spatial coordinates of the i^{th} electron and $\mathbf{R}_A$ the spatial coordinates of the A^{th} nucleus with atomic number Z_A. The two-electron repulsion is

$$\hat{g} = \frac{1}{2}\sum_{i=1}^{N}\sum_{j\neq i}^{N} \frac{1}{|\mathbf{r}_i - \mathbf{r}_j|},$$

and the nucleus-nucleus repulsion is the constant

$$v_{nn} = \frac{1}{2}\sum_{A=1}^{N_n}\sum_{B\neq A}^{N_n} \frac{Z_A Z_B}{|\mathbf{R}_A - \mathbf{R}_B|}.$$

The electronic Schrödinger equation is solved,

$$\hat{H}\Psi_j = E_j\Psi_j \quad j = 0, 1, 2, \ldots,$$

where $\Psi_j(\mathbf{x}_1, \ldots, \mathbf{x}_N)$ is an N-electron wavefunction with $\mathbf{x}_i = (\mathbf{r}_i, s_i)$ denoting the spatial and spin coordinates of electron i [1].

In [2]:

```
Image('1_1.png')
```

Out[2]:

Task I.1: The H_2 molecule.

Consider a H_2 molecule with bond length R_{HH}.

(a) Give a sketch defining the Cartesian coordinates of the nuclei and electrons.

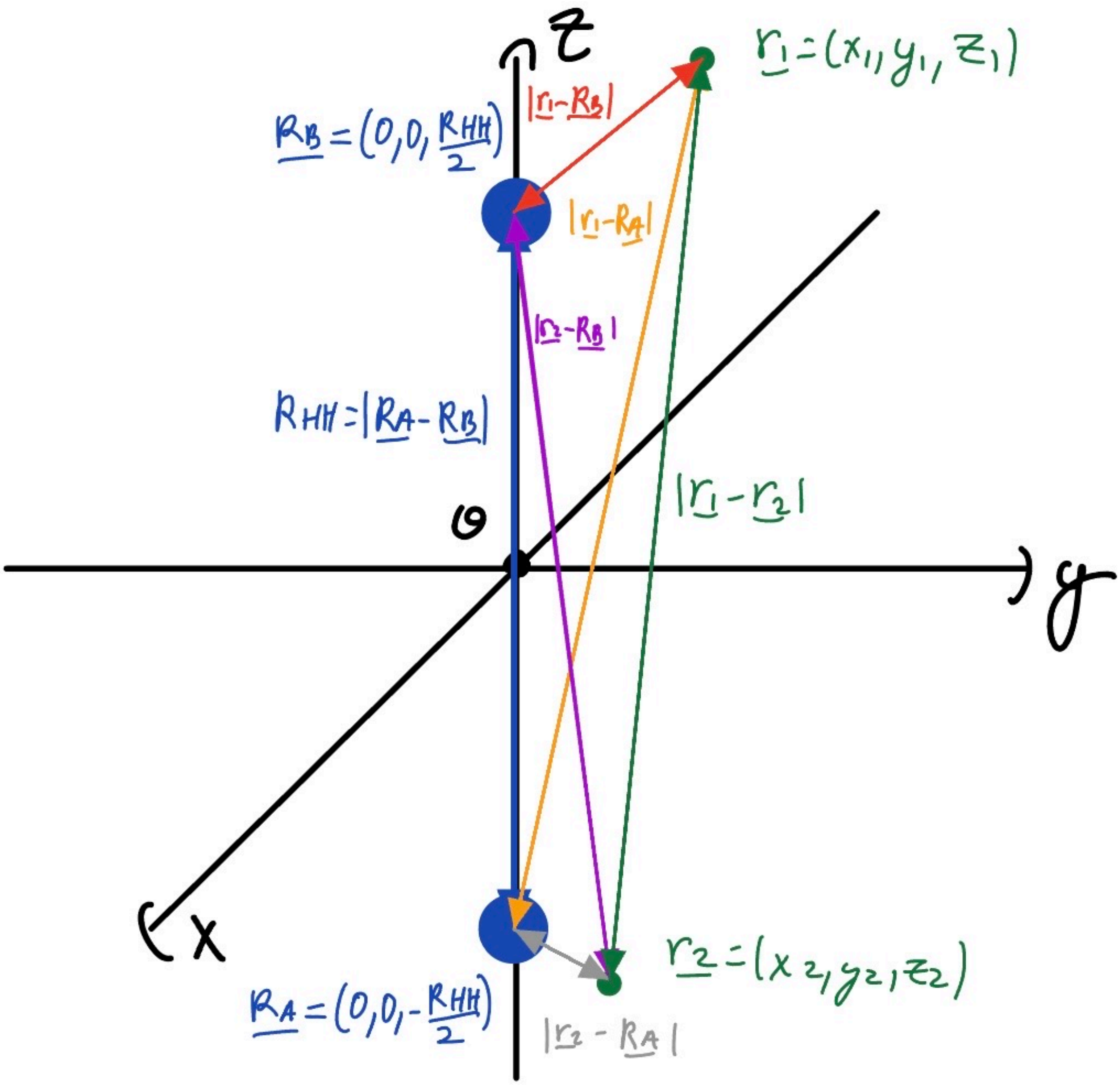
(b) Write down the first-quantised electronic Hamiltonian for this system.

In [3]:

```
Image('1_1_a.jpg')
```

Out[3]:

(a)



(b)

$Z_A = Z_B = 1$

$2(1/2) = 1$

$$\hat{\mathcal{H}} = -\frac{1}{2}\left(\nabla_1^2 + \nabla_2^2\right) - \left(\frac{1}{|\underline{r}_1 - \underline{R}_A|} + \frac{1}{|\underline{r}_1 - \underline{R}_B|} + \frac{1}{|\underline{r}_2 - \underline{R}_A|} + \frac{1}{|\underline{r}_2 - \underline{R}_B|}\right) + \frac{1}{|\underline{r}_1 - \underline{r}_2|} + \frac{1}{|\underline{R}_A - \underline{R}_B|}$$

only real simplification is changing denominator to RHH

Introducing an atomic-orbital basis set

The coordinate representation of $\hat{H}$ and Ψ is typically replaced by a basis-set representation to enable standard linear-algebra methods. For molecules, an atomic-orbital (AO) Gaussian basis $\{\phi_\mu(\mathbf{r})\}$ is used, with each function centred on a nucleus.

The one-electron operator is represented by the core Hamiltonian matrix h with elements

$$h_{\mu\nu} = \int \phi_\mu^*(\mathbf{r})\hat{h}_1(\mathbf{r})\phi_\nu(\mathbf{r})d\mathbf{r} \equiv \langle\mu|\hat{h}_1|\nu\rangle_\mathbf{r}. \quad (1)$$

The two-electron operator is represented by the electron repulsion integral (ERI) tensor g with elements

$$g_{\mu\nu\mu'\nu'} = \iint \phi_\mu^*(\mathbf{r}_1)\phi_\nu^*(\mathbf{r}_2)\hat{g}_{12}(\mathbf{r}_1,\mathbf{r}_2)\phi_{\mu'}(\mathbf{r}_1)\phi_{\nu'}(\mathbf{r}_2)d\mathbf{r}_1 d\mathbf{r}_2 \equiv (\mu\mu'|\nu\nu')_{\mathbf{r}_1\mathbf{r}_2}, \tag{2}$$

with $\hat{g}_{12}(\mathbf{r}_1,\mathbf{r}_2) = 1/r_{12} = 1/|\mathbf{r}_1 - \mathbf{r}_2|$, and using chemists' notation [1].

In [4]:
```python
Image('1_2.png')
```

Out[4]:

Task I.2: The H_2 molecule in a minimal basis.

(a) Write code to output the core Hamiltonian integral matrix $\boldsymbol{h}$ and the ERI array $\mathbf{g}$ calculated by an electronic-structure package of your choice for an input molecular geometry and atomic-orbital basis set.

(b) The most common minimal basis set for molecular systems is STO-3G.

(i) Use the code written in (a) to calculate $\boldsymbol{h}$ and $\mathbf{g}$ in STO-3G for the H_2 molecule defined in Task I.1 with bond length $R_{\mathrm{HH}} = 1$ Å. Make sure to specify whether $\mathbf{g}$ is given in physicists' or chemists' convention.

(ii) By varying the input coordinates, determine whether or not the values of $\boldsymbol{h}$ and $\mathbf{g}$ depend on the orientation of the H_2 molecule in the Cartesian coordinate system. Does the answer still hold if a larger basis set is used, or if the hydrogen atoms are replaced by other atoms?

(a)

In [5]:
```python
def core_and_eri(atom,basis="sto-3g",unit="Angstrom",charge= 0,spin=0):

    mol = gto.Mole()
    mol.atom = atom
    mol.unit = unit
    mol.basis = basis
    mol.charge = charge
    mol.spin = spin
    mol.build()

    # Core Hamiltonian
    h = mol.intor("int1e_kin") + mol.intor("int1e_nuc") #one electron AO integrals

    # PySCF defaults to chemists' notation
    g = mol.intor("int2e")
    v = mol.energy_nuc()#needed later
    return h,g,v
```

(b)

(i)

STO-3G is a minimal basis set where each basis function is a contracted Gaussian made from three Gaussians, fitted to approximate a Slater type orbital. In a minimal basis, one contracted function is used for each occupied atomic orbital of the free atom [2]. For H, the basis used here contains only $s-$type functions on each atom, so each H contributes a single contracted $1s$ basis function.

PySCF has $g_{\mu\mu'\nu\nu'} = (\mu\mu'|\nu\nu')$ [3].

In [6]:
```python
atom = """
H 0 0 0
H 0 0 1
"""
# first is H at origin, second is H at (0,0,1)

print("Chemists' convention")
h,g,v=core_and_eri(atom=atom)
print("\nh =",h)
print("g =",g)
```

```
Chemists' convention

h = [[-0.97949638 -0.68286493]
 [-0.68286493 -0.97949638]]
g = [[[[0.77460594 0.30930897]
   [0.30930897 0.47804137]]

  [[0.30930897 0.15786578]
   [0.15786578 0.30930897]]]


 [[[0.30930897 0.15786578]
   [0.15786578 0.30930897]]

  [[0.47804137 0.30930897]
   [0.30930897 0.77460594]]]]
```

One electron energy in AO $1s$ on each H.

$$h_{11} = (1|\hat{h}|1)_{\mathbf{r}} = -0.9794963769$$

Coupling between the two AOs.

$$h_{12} = (1|\hat{h}|2)_{\mathbf{r}} = -0.6828649331$$

Coulomb repulsion for two electrons in AO 1 (opposite spins).

$$g_{1111} = (11|11)_{\mathbf{r}_1\mathbf{r}_2} = 0.7746059439$$

Coulomb repulsion between the overlap densities.

$$g_{1212} = (12|12)_{\mathbf{r}_1\mathbf{r}_2} = 0.1578657779$$

Coulomb interaction between densities on different atoms.

$$g_{1122} = (11|22)_{\mathbf{r}_1\mathbf{r}_2} = 0.4780413732$$

Exchange interaction.

$$g_{1221} = (12|21)_{\mathbf{r}_1\mathbf{r}_2} = 0.1578657779$$

Repulsion between the density on atom 1 and the overlap density.

$$g_{1112} = (11|12)_{\mathbf{r}_1\mathbf{r}_2} = 0.3093089672$$

(ii)

1. Along the $z-$axis, centered at the origin.

In [7]:
```python
atom = """
H 0 0 -0.5
H 0 0 0.5
"""

h,g,v=core_and_eri(atom=atom)
print("h =",h)
print("g =",g)
```

```
h = [[-0.97949638 -0.68286493]
 [-0.68286493 -0.97949638]]
g = [[[[0.77460594 0.30930897]
   [0.30930897 0.47804137]]

  [[0.30930897 0.15786578]
   [0.15786578 0.30930897]]]


 [[[0.30930897 0.15786578]
   [0.15786578 0.30930897]]

  [[0.47804137 0.30930897]
   [0.30930897 0.77460594]]]]
```

Along $y-$axis, with $z = 1$.

In [8]:
```python
atom = """
H 0 1 1
```

```
H 0 0 1
"""
h,g,v=core_and_eri(atom=atom)
print("h =",h)
print("g =",g)
```

```
h = [[-0.97949638 -0.68286493]
 [-0.68286493 -0.97949638]]
g = [[[[0.77460594 0.30930897]
   [0.30930897 0.47804137]]

  [[0.30930897 0.15786578]
   [0.15786578 0.30930897]]]


 [[[0.30930897 0.15786578]
   [0.15786578 0.30930897]]

  [[0.47804137 0.30930897]
   [0.30930897 0.77460594]]]]
```

At 45 degrees in the $xy-$plane.

In [9]:
```
atom = """
H -0.353553 -0.353553 0
H  0.353553  0.353553 0
"""
h,g,v=core_and_eri(atom=atom)
print("h =",h)
print("g =",g)
```

```
h = [[-0.97949687 -0.68286598]
 [-0.68286598 -0.97949687]]
g = [[[[0.77460594 0.30930948]
   [0.30930948 0.47804173]]

  [[0.30930948 0.15786624]
   [0.15786624 0.30930948]]]


 [[[0.30930948 0.15786624]
   [0.15786624 0.30930948]]

  [[0.47804173 0.30930948]
   [0.30930948 0.77460594]]]]
```

Small differences due to numerical roundoff.

Along $(1, 1, 1)$ direction

In [10]:
```
atom = """
H -0.288675 -0.288675 -0.288675
H  0.288675  0.288675  0.288675
"""
h,g,v=core_and_eri(atom=atom)
print("h =",h)
print("g =",g)
```

```
h = [[-0.97949659 -0.68286537]
 [-0.68286537 -0.97949659]]
g = [[[[0.77460594 0.30930918]
   [0.30930918 0.47804153]]

  [[0.30930918 0.15786597]
   [0.15786597 0.30930918]]]


 [[[0.30930918 0.15786597]
   [0.15786597 0.30930918]]

  [[0.47804153 0.30930918]
   [0.30930918 0.77460594]]]]
```

The electronic Hamiltonian is invariant under rigid rotations, so any observable is independent of the molecule's absolute orientation, but the AO integral tensors h_{pq} and g_{pqrs} are basis dependent. If the chosen AO basis **contains $l > 0$ functions** (even if the occupied atomic orbitals are $s-$type), a rigid rotation mixes AOs within each angular momentum shell, so the rotated integrals are generally not equal elementwise, but are related by the corresponding AO rotation within each shell.

H_2 is invariant here because the chosen basis is effectively $s-$only, so rotation doesn't mix AOs.

Consider Li_2, Li's occupied orbitals have $l = 0$ (including an occupied $2s$ valence orbital), and since $2s$ and $2p$ orbitals share the same exponents, the **basis** contains $l > 0$ (p) functions [4]. Since the chosen basis includes p functions, the rotation mixes (p_x, p_y, p_z) and changes h, g elementwise. Let's verify this.

Start with the first Li at the origin, and the second along the $z-$axis up to $z = 1$.

In [166…
```python
atom = """
Li 0 0 0
Li 0 0 1
"""
h,g,v=core_and_eri(atom=atom)
print("h shape:", h.shape)
print("g shape:", g.shape)
print("\nTop-left 5x5 block of h:")
print(h[:5, :5])

print("\nRandom g elements:")
inds = [(0,0,0,0),(0,0,1,1),(0,0,2,7)]
for i,j,k,l in inds:
    print(f"g[{i},{j},{k},{l}] = {g[i,j,k,l]:f}")
```

```
h shape: (10, 10)
g shape: (10, 10, 10, 10)

Top-left 5x5 block of h:
[[-5.99338978 -1.363622    0.          0.         -0.11716946]
 [-1.363622   -2.17419742  0.          0.         -0.42176859]
 [ 0.          0.         -1.83933536  0.          0.        ]
 [ 0.          0.          0.         -1.83933536  0.        ]
 [-0.11716946 -0.42176859  0.          0.         -2.16072041]]

Random g elements:
g[0,0,0,0] = 1.680395
g[0,0,1,1] = 0.397727
g[0,0,2,7] = 0.287511
```

At 45 degrees in the $xy-$plane.

In [167…
```python
atom = """
Li -0.353553 -0.353553 0
Li  0.353553  0.353553 0
"""
h,g,v=core_and_eri(atom=atom)
print("h shape:", h.shape)
print("g shape:", g.shape)
print("\nTop-left 5x5 block of h:")
print(h[:5, :5])

print("\nRandom g elements:")
for i,j,k,l in inds:
    print(f"g[{i},{j},{k},{l}] = {g[i,j,k,l]:f}")
```

```
h shape: (10, 10)
g shape: (10, 10, 10, 10)

Top-left 5x5 block of h:
[[-5.99339153 -1.36362239 -0.08285147 -0.08285147  0.        ]
 [-1.36362239 -2.17419775 -0.29823541 -0.29823541  0.        ]
 [-0.08285147 -0.29823541 -2.0000282  -0.16069247  0.        ]
 [-0.08285147 -0.29823541 -0.16069247 -2.0000282   0.        ]
 [ 0.          0.          0.          0.         -1.83933573]]

Random g elements:
g[0,0,0,0] = 1.680395
g[0,0,1,1] = 0.397727
g[0,0,2,7] = 0.203928
```

As expected, both are different. The size of h is 10×10, i.e. 10 spatial AOs in total, or 5 per Li. This indicates that the basis includes p functions, with each Li contributing one s function for $1s$, one s function for $2s$, and three p functions ($2p_x, 2p_y, 2p_z$).

Second quantisation

For quantum-computing treatments of electronic structure, the coordinate representation is replaced by second quantisation, where the Hamiltonian and states are expressed using discrete fermionic creation and annihilation operators. This naturally enforces antisymmetry and provides a direct encoding in an occupation-number basis.

Given M orthonormal spin-orbitals $\{\chi_p(\mathbf{x})\}_{p=0}^{M-1}$ with $\mathbf{x} = (\mathbf{r}, s)$, an N-electron configuration is represented by an occupation vector

$$|\mathbf{k}\rangle = |k_0, k_1, \ldots, k_{M-1}\rangle, \qquad k_p \in \{0, 1\}.$$

The elementary fermionic operators change the occupation of a given spin-orbital, with the fermionic phase factor $\Gamma_p^{\mathbf{k}} = \prod_{q=0}^{p-1} (-1)^{k_q}$ [1],

$$\hat{a}_p^\dagger | \ldots, k_p, \ldots \rangle = \delta_{k_p,0} \Gamma_p^{\mathbf{k}} | \ldots, 1, \ldots \rangle,$$

$$\hat{a}_p | \ldots, k_p, \ldots \rangle = \delta_{k_p,1} \Gamma_p^{\mathbf{k}} | \ldots, 0, \ldots \rangle.$$

In [13]:
```
Image('1_3.png')
```

Out[13]:

Task I.3: Data structure and algebras for fermionic operators.

This task may also be done with the aid of appropriate operator classes in OpenFermion.

(a) Program a data structure entitled `FermionicOperator` to represent the elementary fermionic operators for the p^{th} spin-orbital, $\hat{a}_p^\dagger$ and $\hat{a}_p$.

(b) Extend `FermionicOperator` to handle an arbitrary string of elementary fermionic operators, *e.g.* $\hat{a}_1^\dagger \hat{a}_2^\dagger \hat{a}_3 \hat{a}_4$.

(c) Further extend `FermionicOperator` to handle *multiple* additive strings of elementary fermionic operators, each of which is accompanied by a complex coefficient, *e.g.* $0.5\hat{a}_1^\dagger \hat{a}_2^\dagger \hat{a}_3 \hat{a}_4 + 0.2\hat{a}_2^\dagger \hat{a}_1^\dagger \hat{a}_4 \hat{a}_3$. Note that the order of the elementary fermionic operators in each term is important, so the two terms in the given example cannot be trivially combined without recourse to *anticommutation relations*, which we will not cover in this course.

(d) Implement the additive and multiplicative complex algebras for `FermionicOperator` so that multiple `FermionicOperator` instances can be added or multiplied together to give another `FermionicOperator` instance. For example,

- $0.5\hat{a}_1^\dagger \hat{a}_2^\dagger \hat{a}_3 \hat{a}_4 + (-0.2)\hat{a}_1^\dagger \hat{a}_2^\dagger \hat{a}_3 \hat{a}_4 = 0.3\hat{a}_1^\dagger \hat{a}_2^\dagger \hat{a}_3 \hat{a}_4$,
- $0.5\hat{a}_1^\dagger \hat{a}_2^\dagger \times 0.5\hat{a}_2 \hat{a}_3 = 0.25\hat{a}_1^\dagger \hat{a}_2^\dagger \hat{a}_2 \hat{a}_3$.

In [14]:
```
class FermionicOperator:
    def __init__(self, _of=None):
        self._of = _of if _of is not None else FO()
#stores operator, if empty allows zero operator

#(a)
    @staticmethod #(does not use self)
    def create(i):
        return FermionicOperator(FO(((int(i), 1),), 1.0)) #create at site i (coeff of 1)
#((int(i), 1),) is a one-element tuple holding (int(i), 1)
    @staticmethod
    def destroy(i):
        return FermionicOperator(FO(((int(i), 0),), 1.0))

#(b)
    @staticmethod
    def string(ops, coeff=1.0):
        term = tuple((int(i),ca) for i, ca in ops)
#ops is an ordered list of (i, ca), ca is create/annihilate (1 or 0 respectively)
        return FermionicOperator(FO(term, complex(coeff)))

#(c)
    @staticmethod
    def sum(terms):
        of = FO()
```

```
        for coeff, ops in terms:
            of += FermionicOperator.string(ops, coeff)._of #unwraps to get FO
        return FermionicOperator(of)
#(d)
# __add__ adds two FO objects,
    def __add__(self, other):
        return FermionicOperator(self._of + other._of)

    def __mul__(self, other):
        if isinstance(other, FermionicOperator):
# if other is a fermionic operator, do operator multiplication, otherwise do scalar multiplication
            return FermionicOperator(self._of * other._of)
        return FermionicOperator(self._of * complex(other))

    def __rmul__(self, other):
        return FermionicOperator(complex(other) * self._of)

    def __str__(self):
        #useful for II.3 part (d), prints string of object
        return str(self._of)



    def to_openfermion(self):
        return self._of
#returns operator
```

Sum and Add do the same thing in slightly different ways which is shown below, Sum reads slightly cleaner, but Add follows the task closer, so I have included both.

Verify that $0.5\hat{a}_1^\dagger\hat{a}_2^\dagger\hat{a}_3\hat{a}_4 + (-0.2)\hat{a}_1^\dagger\hat{a}_2^\dagger\hat{a}_3\hat{a}_4 = 0.3\hat{a}_1^\dagger\hat{a}_2^\dagger\hat{a}_3\hat{a}_4$.

In [15]:
```
expr1 = FermionicOperator.sum([
    (0.5, [(1,1), (2,1), (3,0), (4,0)]),
    (-0.2, [(1,1), (2,1), (3,0), (4,0)])
])
print('Using Sum, expression becomes',expr1.to_openfermion())

expr2 = (FermionicOperator.string([(1,1), (2,1), (3,0), (4,0)], 0.5)
         + FermionicOperator.string([(1,1), (2,1), (3,0), (4,0)], -0.2))
print('\nUsing Add, expression becomes',expr2.to_openfermion())
```

```
Using Sum, expression becomes (0.3+0j) [1^ 2^ 3 4]

Using Add, expression becomes (0.3+0j) [1^ 2^ 3 4]
```

Verify that $0.5\hat{a}_1^\dagger\hat{a}_2^\dagger \times 0.5\hat{a}_2\hat{a}_3 = 0.25\hat{a}_1^\dagger\hat{a}_2^\dagger\hat{a}_2\hat{a}_3$

In [16]:
```
expr3 = (FermionicOperator.string([(1,1), (2,1)], 0.5)
         * FermionicOperator.string([(2,0), (3,0)], 0.5))
print('Expression becomes',expr3.to_openfermion())
```

```
Expression becomes (0.25+0j) [1^ 2^ 2 3]
```

Module 1 Logbook

The laboratory introduced the electronic Hamiltonian for an N electron system with N_n fixed nuclei, written as $\hat{\mathcal{H}} = \hat{h} + \hat{g} + v_{nn}$, where $\hat{h}$ is the one-electron term, $\hat{g}$ the two-electron interaction, and v_{nn} the nucleus-nucleus repulsion. The coordinate representation was introduced first, then the Hamiltonian was expressed in an AO basis through the integrals $h_{\mu\nu} = \langle\mu|\hat{h}_1|\nu\rangle_{\mathbf{r}}$ and $g_{\mu\mu'\nu\nu'} = (\mu\mu'|\nu\nu')$ used in practical calculations. For H_2 with bond length $R_{HH} = 1$ Angstrom in the STO-3G minimal basis, the core Hamiltonian integral matrix $\mathbf{h}$ and the electron repulsion integral array $\mathbf{g}$ were generated using PySCF. STO-3G is a minimal Gaussian basis set in which each basis function is formed by contracting three Gaussian primitives to approximate a Slater-type orbital. In a minimal basis, it assigns a single contracted function to each occupied atomic orbital of the isolated atom [2]. The output convention for $\mathbf{g}$ was the chemists' convention $g_{\mu\mu'\nu\nu'} = (\mu\mu'|\nu\nu')$.

Orientation tests agree with theory. For H_2, $\mathbf{h}$ and $\mathbf{g}$ are unchanged under rigid rotations and translations, as expected when the AO basis is effectively s only. While physical observables are rotation invariant, AO integrals are only elementwise invariant when the basis contains no $l > 0$ functions. Once p (or higher) functions are present, a rigid rotation mixes (p_x, p_y, p_z) within each atom's shell, so h_{pq} and g_{pqrs} generally change elementwise, as seen in the Li_2 check.

Second quantisation was then introduced, treating electrons as excitations in a spin-orbital basis, which simplifies many-body problems. Instead of working with continuous coordinates, the system is described in a discrete set of spin orbitals using fermionic creation and annihilation operators. The occupation number picture makes the state space explicit, and the fermionic operator algebra automatically enforces antisymmetry. A FermionicOperator data structure was implemented to represent these creation and annihilation operators, ordered strings of operators, and sums of these strings with complex coefficients.

Correctness was checked in two ways. The additive and multiplicative examples provided in the laboratory book were reproduced, confirming that identical strings combined by coefficient addition and that multiplication produced the expected coefficient product with the operator strings concatenated in the correct order. For the integral generation, the rotation invariance test confirmed that the geometry, units, and integral calls were consistent.

Part II

Towards Quantum Computing

The occupation-number state $|\mathbf{k}\rangle$ is generated from the vacuum by [1]

$$|\mathbf{k}\rangle = \left[\prod_{p=0}^{M-1} \left(\hat{a}_p^\dagger\right)^{k_p}\right] |\text{vac}\rangle = \left(\hat{a}_0^\dagger\right)^{k_0} \left(\hat{a}_1^\dagger\right)^{k_1} \cdots \left(\hat{a}_{M-1}^\dagger\right)^{k_{M-1}} |0, 0, \cdots, 0\rangle$$

$$= \begin{cases} \left(\hat{a}_{M-1}^\dagger\right)^1 |0, 0, \cdots, 0\rangle = |0, 0, \cdots, 0, 1\rangle, & k_0 = \cdots = k_{M-2} = 0, k_{M-1} = 1 \\ |0, 0, \cdots, 0\rangle, & k_0 = \cdots = k_{M-1} = 0. \end{cases}$$

For fermions, a fixed mode ordering is applied to avoid sign ambiguities, the product then acts on $|\text{vac}\rangle$ from right to left.

Note that until stated otherwise, the convention that the leftmost bit corresponds to qubit 0 will be used, as in the manual.

In [17]:
```
Image('2_1.png')
```

Out[17]:

Task II.1: Second-quantised Slater determinants.

This task is optional.

Write code that takes in an occupation-number vector and outputs a `FermionicOperator` (Task I.3) that enables its construction symbolically from the vacuum state according to Equation (3.11).

In [18]:
```python
def occupation_to_operator(k):
    ops = [(i, 1) for i, occ in enumerate(k) if occ] #(i,1) for every occ=1
    return FermionicOperator.string(ops, 1.0)
```

In [19]:
```python
k1 = (1,0,1,0)
op = occupation_to_operator(k1)
print('k1 =',op.to_openfermion())


k2 = (0,1,1,0,1)
op2 = occupation_to_operator(k2)
print('k2 =',op2.to_openfermion())
```

```
k1 = (1+0j) [0^ 2^]
k2 = (1+0j) [1^ 2^ 4^]
```

The electronic Hamiltonian is written in second quantisation as

$$\hat{H} = \sum_{pq}^{M-1} h_{pq} \hat{a}_p^\dagger \hat{a}_q + \frac{1}{2} \sum_{pqrs}^{M-1} g_{pqrs} \hat{a}_p^\dagger \hat{a}_r^\dagger \hat{a}_s \hat{a}_q + v_{nn},$$

where the one- and two-electron integrals are

$$h_{pq} = \int \chi_p^*(\mathbf{x}) \hat{h}_1(\mathbf{r}) \chi_q(\mathbf{x}) d\mathbf{x}, \quad g_{pqrs} = \iint \chi_p^*(\mathbf{x}_1) \chi_q^*(\mathbf{x}_2) \hat{g}_{12}(\mathbf{r}_1, \mathbf{r}_2) \chi_r(\mathbf{x}_1) \chi_s(\mathbf{x}_2) d\mathbf{x}_1 d\mathbf{x}_2.$$

Unlike the integrals in (1) and (2), these involve integration over both spatial and spin coordinates, rather than only involving spatial coordinates [1].

In [20]:
```
Image('2_2.png')
```

Out[20]:

Task II.2: Transformation from atomic orbitals to spin-orbitals.

(a) Given the form of the spin-orbitals in Equation (3.2), show that h_{pq} and g_{pqrs} can be written in terms of $h_{\mu\nu}$ and $g_{\mu\nu\mu'\nu'}$ by

$$h_{pq} = \langle\zeta_p|\zeta_q\rangle_s \sum_{\mu\nu} h_{\mu\nu} C^*_{\mu p} C_{\nu q}, \tag{3.15}$$

$$g_{pqrs} = \langle\zeta_p|\zeta_r\rangle_s \langle\zeta_q|\zeta_s\rangle_s \sum_{\mu\nu\mu'\nu'} g_{\mu\nu\mu'\nu'} C^*_{\mu p} C^*_{\nu q} C_{\mu' r} C_{\nu' s}. \tag{3.16}$$

(b) Noting the orthonormality of the spin functions [Equation (3.4)], identify the non-zero spin blocks of (h_{pq}) and (g_{pqrs}).

In [21]:

```
Image('2_2a.jpg')
```

Out[21]:

(a)

$$h_{\mu\nu} = \int \varphi^*_\mu(\underline{r})\, \hat{h}_1(\underline{r})\, \varphi_\nu(\underline{r})\, d\underline{r}\,, \quad h_{pq} = \int \chi^*_p(\underline{x})\, \hat{h}_1(\underline{r})\, \chi_q(\underline{x})\, d\underline{x}$$

$$g_{\mu\nu\mu'\nu'} = \int \varphi^*_\mu(\underline{r_1})\, \varphi^*_\nu(\underline{r_2})\, \hat{g}_{12}(\underline{r_1},\underline{r_2})\, \varphi_{\mu'}(\underline{r_1})\, \varphi_{\nu'}(\underline{r_2})\, d\underline{r_1}\, d\underline{r_2}\,,$$

$$g_{pqrs} = \int \chi^*_p(\underline{x_1})\, \chi^*_q(\underline{x_2})\, \hat{g}_{12}(\underline{r_1},\underline{r_2})\, \chi_r(\underline{x_1})\, \chi_s(\underline{x_2})\, d\underline{x_1}\, d\underline{x_2}$$

$$\chi_p(\underline{x}) = \zeta_p(s) \sum_\mu \varphi_\mu(\underline{r}) C_{\mu p}\,, \quad \int \zeta_p(s)\, \zeta_q(s)\, ds = \langle \zeta_p | \zeta_q \rangle_s = \delta_{\zeta_p \zeta_q}$$

$$\Rightarrow h_{pq} = \int \left(\sum_\mu C^*_{\mu p}\, \varphi^*_\mu(\underline{r})\right) \zeta^*_p(s)\, \hat{h}_1(\underline{r})\, \zeta_q(s) \left[\sum_\nu \varphi_\nu(\underline{r}) C_{\nu q}\right] d\underline{x}$$

$$= \int ds \int d\underline{r} \left(\sum_{\mu\nu} C^*_{\mu p}\, \varphi^*_\mu(\underline{r})\, \zeta^*_p(s)\, \hat{h}_1(\underline{r})\, \zeta_q(s)\, \varphi_\nu(\underline{r}) C_{\nu q}\right)$$

$$= \int ds\, \zeta^*_p(s)\, \zeta_q(s) \int d\underline{r} \Bigl(\underbrace{\sum_{\mu\nu} C^*_{\mu p}\, \varphi^*_\mu(\underline{r})\, \hat{h}_1(\underline{r})\, \varphi_\nu(\underline{r}) C_{\nu q}}_{\text{spin independent}}\Bigr)$$

$$= \langle \zeta_p | \zeta_q \rangle_s \sum_{\mu\nu} C^*_{\mu p} \int d\underline{r} \left(\varphi^*_\mu(\underline{r})\, \hat{h}_1(\underline{r})\, \varphi_\nu(\underline{r})\right) C_{\nu q}$$

$$= \langle \zeta_p | \zeta_q \rangle_s \sum_{\mu\nu} C^*_{\mu p}\, h_{\mu\nu}\, C_{\nu q}$$

$$\Rightarrow h_{pq} = \langle \zeta_p | \zeta_q \rangle_s \sum_{\mu\nu} h_{\mu\nu} C^*_{\mu p} C_{\nu q}\,, \text{ as desired!}$$

$$g_{pqrs} = \int ds_1 ds_2 \int d\underline{r_1} d\underline{r_2} \Bigl(\sum_\mu C^*_{\mu p}\, \varphi^*_\mu(\underline{r_1})\, \zeta^*_p(s_1) \sum_\nu C^*_{\nu q}\, \varphi^*_\nu(\underline{r_2})\, \zeta^*_q(s_2)\, \hat{g}_{12}(\underline{r_1},\underline{r_2})$$
$$\zeta_r(s_1) \sum_{\mu'} \varphi_{\mu'}(\underline{r_1}) C_{\mu' r}\, \zeta_s(s_2) \sum_{\nu'} \varphi_{\nu'}(\underline{r_2}) C_{\nu' s} \Bigr)$$

same variable!

scalar

$$= \int ds_1 ds_2\, \zeta^*_p(s_1)\, \underbrace{\zeta^*_q(s_2)\, \zeta_r(s_1)}_{\text{dif variables}}\, \zeta_s(s_2) \sum_{\mu\nu\mu'\nu'} \int d\underline{r_1} d\underline{r_2}\, C^*_{\mu p}\, \varphi^*_\mu(\underline{r_1})\, C^*_{\nu q}\, \varphi^*_\nu(\underline{r_2})\, \hat{g}_{12}(\underline{r_1},\underline{r_2})\, \varphi_{\mu'}(\underline{r_1}) C_{\mu' r}\, \varphi_{\nu'}(\underline{r_2}) C_{\nu' s}$$

$$\Rightarrow g_{pqrs} = \langle \zeta_p | \zeta_r \rangle_s \langle \zeta_q | \zeta_s \rangle_s \sum_{\mu\nu\mu'\nu'} g_{\mu\nu\mu'\nu'} C^*_{\mu p} C^*_{\nu q} C_{\mu' r} C_{\nu' s}\,, \text{ as desired!}$$

In [22]:
```
Image('2_2b.jpg')
```

Out[22]:

(b)

$$\langle \zeta_p | \zeta_q \rangle_s = \delta_{\zeta_p \zeta_q}, \quad \zeta_p(s) = \begin{cases} \alpha(s) \\ \beta(s) \end{cases}$$

$$h_{pq} = \delta_{\zeta_p \zeta_q} \sum_{\mu\nu} h_{\mu\nu} C_{\mu p}^* C_{\nu q} \Rightarrow \text{only } h_{\alpha\alpha} \text{ and } h_{\beta\beta} \text{ survive}$$

$$g_{pqrs} = \delta_{\zeta_p \zeta_r} \delta_{\zeta_q \zeta_s} \sum_{\mu\nu\mu'\nu'} g_{\mu\nu\mu'\nu'} C_{\mu p}^* C_{\nu q}^* C_{\mu' r} C_{\nu' s}, \text{ as desired!}$$

$\Rightarrow$ only $g_{\alpha\alpha\alpha\alpha}$, $g_{\alpha\beta\alpha\beta}$, $g_{\beta\beta\beta\beta}$, and $g_{\beta\alpha\beta\alpha}$ survive

In [23]:
```
Image('2_3a.png')
```

Out[23]:

Task II.3: Second-quantised Hamiltonian for the H_2 molecule.

(a) Consult an electronic-structure textbook to understand the differences between restricted Hartree–Fock (RHF) and unrestricted Hartree–Fock (UHF).

(b) Given an input molecular geometry and atomic-orbital basis set, write code to run a UHF calculation using an electronic-structure package of choice and

In [24]:
```
Image('2_3b.png')
```

Out[24]:

obtain the UHF MOs in the form of Equation (3.2).

(c) From the obtained UHF MOs, the results of Task I.2(a) and Task II.2, and the `FermionicOperator` data structure from Task I.3, extend the code to construct the second-quantised Hamiltonian defined in Equation (3.12). In particular, your code should

- have a clear definition of the spin projection of each spin-orbital, *e.g.* whether the spin part of the spin-orbital χ_p corresponding to $\hat{a}_p^\dagger$ or $\hat{a}_p$ is α or β;
- be able to store and output the symbolic representation of all non-zero terms in the second-quantised electronic Hamiltonian; and
- have a sensible grouping of these terms according to the non-zero spin blocks identified in Task II.2(b).

(d) Use this code to construct the second-quantised electronic Hamiltonian for the H_2 molecule defined in Task I.1 with bond length $R_{HH} = 1\,\text{Å}$.

(a)

Restricted Hartree-Fock (RHF) uses the same spatial orbital for the α and β electrons in each pair. It works best for closed-shell molecules, however since the same spatial orbital is used, it cannot describe bond breaking into open-shell molecules/atoms. E.g. H_2 dissociates into H^+ and H^-, rather than two H atoms.

Unrestricted Hartree-Fock (UHF) uses separate spatial orbitals for α and β electrons. UHF can be used for open and closed shell molecules, and usually dissociates correctly. At equilibrium geometry for closed shell molecules, RHF and UHF wavefunctions are almost identical. RHF wavefunctions are usually used for closed shells, because it is cheaper, however it shouldn't be used for closed shells that may dissociate into open shells. Separate spatial orbitals introduce 'spin contamination' error, whose size depends on the chemical system involved [2].

(b)

In [25]:
```
def uhf_mos(atom, basis, unit="Angstrom", charge=0, spin=0):

    mol = gto.Mole()
    mol.atom = atom
    mol.unit = unit
    mol.basis = basis
    mol.charge = charge
    mol.spin = spin
    mol.build()


    mf = scf.UHF(mol)
    mf.verbose = 0 #no print
    mf.run()
    occ_alpha, occ_beta = mf.mo_occ
    E_uhf = mf.e_tot

    C_alpha, C_beta = mf.mo_coeff
    ao_labels = mol.ao_labels()
    #all of these will be useful for later sections
    return C_alpha, C_beta, ao_labels, (occ_alpha, occ_beta), E_uhf
```

(c)

Note 1

Noting that $\sum_{\mu\nu} h_{\mu\nu} C^*_{\mu p} C_{\nu q} = \sum_\mu C^*_{\mu p} \sum_\nu h_{\mu\nu} C_{\nu q} = \sum_\mu \left(C^\dagger\right)_{p\mu} (hC)_{\mu q} = \left(C^\dagger h C\right)_{pq}$

Note 2

Noting the einstein summation used to find g_{pqrs}.

In [26]:
```
a = np.arange(16).reshape(2,2,2,2)
b = np.arange(4).reshape(2,2)
print(a)
print(b)
```

```
[[[[ 0  1]
   [ 2  3]]

  [[ 4  5]
   [ 6  7]]]


 [[[ 8  9]
   [10 11]]

  [[12 13]
   [14 15]]]]
[[0 1]
 [2 3]]
```

In [27]:
```
Image('note_2.jpg')
```

Out[27]:

$$a = \begin{pmatrix} \begin{pmatrix} 0 & 1 \\ 2 & 3 \end{pmatrix} & \begin{pmatrix} 4 & 5 \\ 6 & 7 \end{pmatrix} \\ \begin{pmatrix} 8 & 9 \\ 10 & 11 \end{pmatrix} & \begin{pmatrix} 12 & 13 \\ 14 & 15 \end{pmatrix} \end{pmatrix}, \quad b = \begin{pmatrix} 0 & 1 \\ 2 & 3 \end{pmatrix}$$

$$C_{0000} = \sum_{\mu\nu\mu'\nu'} a_{\mu\nu\mu'\nu'} b_{\mu 0} b_{\nu 0} b_{\mu' 0} b_{\nu' 0}, \quad b_{00} = 0, \; b_{10} = 2$$

$\Rightarrow$ only non-zero component is $\mu = \nu = \mu' = \nu' = 1$

$$\Rightarrow C_{0000} = a_{1111} (2)^4 = 15 (2)^4 = 240$$

In [28]:

```python
c=np.einsum('abcd,ap,bq,cr,ds->pqrs', a, b, b, b, b)
print(c)
```

```
[[[[ 240  472]
   [ 464  912]]

  [[ 448  880]
   [ 864 1696]]]


 [[[ 416  816]
   [ 800 1568]]

  [[ 768 1504]
   [1472 2880]]]]
```

$\Rightarrow g_{pqrs} = \sum_{\mu\nu\mu'\nu'} g_{\mu\nu\mu'\nu'} C^*_{\mu p} C^*_{\nu q} C_{\mu' r} C_{\nu' s}$ for the four cases identified in II.2(b).

In [29]:

```python
def second_quantized_hamiltonian(atom, basis, unit = "Angstrom", charge = 0, spin = 0):

    C_alpha, C_beta, ao_labels, (occ_alpha, occ_beta),E_uhf=uhf_mos(atom,basis,unit,charge, spin)
    h,g,v=core_and_eri(atom,basis,unit,charge,spin)

    # Note 1 above
    h_aa = C_alpha.conj().T @ h @ C_alpha
    h_bb = C_beta.conj().T  @ h @ C_beta

    # Note 2 above
    g_aaaa = np.einsum('abcd,ap,bq,cr,ds->pqrs', g, C_alpha.conj(), C_alpha.conj(), C_alpha, C_alpha)
    g_bbbb = np.einsum('abcd,ap,bq,cr,ds->pqrs', g, C_beta.conj(),  C_beta.conj(),  C_beta,  C_beta)
    g_abab = np.einsum('abcd,ap,bq,cr,ds->pqrs', g, C_alpha.conj(), C_beta.conj(),  C_alpha, C_beta)
    g_baba = np.einsum('abcd,ap,bq,cr,ds->pqrs', g, C_beta.conj(),  C_alpha.conj(), C_beta,  C_alpha)

    h_αα=FermionicOperator()
    h_ββ=FermionicOperator()
    g_αααα=FermionicOperator()
    g_αβαβ=FermionicOperator()
    g_βαβα=FermionicOperator()
    g_ββββ=FermionicOperator()

    nmo = h_aa.shape[0]  # number of spatial molecular orbitals

    for p in range(nmo):
        for q in range(nmo):
            #one-electron terms
            h_αα += FermionicOperator.string([(2*p, 1), (2*q, 0)], h_aa[p, q])
            # need different indexing since a_α neq a_β, have α even, β odd
            h_ββ += FermionicOperator.string([(2*p+1, 1), (2*q+1, 0)], h_bb[p, q])
            for r in range(nmo):
                for s in range(nmo):
                    #two-electron terms
                    #swap q and r as using chemist notation
```

```
                    g_αααα+=FermionicOperator.string([(2*p, 1), (2*q, 1), (2*s, 0), (2*r, 0)],g_aaaa[p,r,q,s])
                    g_αβαβ+=FermionicOperator.string([(2*p, 1), (2*q+1, 1), (2*s+1, 0), (2*r, 0)],g_abab[p,r,q,
                    g_βαβα+=FermionicOperator.string([(2*p+1, 1), (2*q, 1), (2*s, 0), (2*r+1, 0)],g_baba[p,r,q,
                    g_ββββ+=FermionicOperator.string([(2*p+1, 1), (2*q+1, 1), (2*s+1, 0), (2*r+1, 0)],g_bbbb[p,

    H = h_αα + h_ββ + 0.5*(g_αααα + g_αβαβ + g_βαβα + g_ββββ) + FermionicOperator.string([], v)#need constant c
    return H
```

(d)

In [30]:

```
atom = """
H 0 0 0
H 0 0 1
"""
basis = "sto-3g"

H = second_quantized_hamiltonian(atom=atom, basis="sto-3g")
print(H)
```

```
(0.52917721092+0j) [] +
(-1.110844179883727+0j) [0^ 0] +
(0.3132012497647588+0j) [0^ 0^ 0 0] +
(0.09839529174273509+0j) [0^ 0^ 2 2] +
(0.3132012497647588+0j) [0^ 1^ 1 0] +
(0.09839529174273509+0j) [0^ 1^ 3 2] +
(0.09839529174273508+0j) [0^ 2^ 0 2] +
(0.3108533815598566+0j) [0^ 2^ 2 0] +
(0.09839529174273508+0j) [0^ 3^ 1 2] +
(0.3108533815598566+0j) [0^ 3^ 3 0] +
(0.3132012497647588+0j) [1^ 0^ 0 1] +
(0.09839529174273509+0j) [1^ 0^ 2 3] +
(-1.110844179883727+0j) [1^ 1] +
(0.3132012497647588+0j) [1^ 1^ 1 1] +
(0.09839529174273509+0j) [1^ 1^ 3 3] +
(0.09839529174273508+0j) [1^ 2^ 0 3] +
(0.3108533815598566+0j) [1^ 2^ 2 1] +
(0.09839529174273508+0j) [1^ 3^ 1 3] +
(0.3108533815598566+0j) [1^ 3^ 3 1] +
(0.31085338155985676+0j) [2^ 0^ 0 2] +
(0.09839529174273512+0j) [2^ 0^ 2 0] +
(0.31085338155985676+0j) [2^ 1^ 1 2] +
(0.09839529174273512+0j) [2^ 1^ 3 0] +
(-0.5891210037060842+0j) [2^ 2] +
(0.0983952917427351+0j) [2^ 2^ 0 0] +
(0.3265353734712868+0j) [2^ 2^ 2 2] +
(0.0983952917427351+0j) [2^ 3^ 1 0] +
(0.3265353734712868+0j) [2^ 3^ 3 2] +
(0.31085338155985676+0j) [3^ 0^ 0 3] +
(0.09839529174273512+0j) [3^ 0^ 2 1] +
(0.31085338155985676+0j) [3^ 1^ 1 3] +
(0.09839529174273512+0j) [3^ 1^ 3 1] +
(0.0983952917427351+0j) [3^ 2^ 0 1] +
(0.3265353734712868+0j) [3^ 2^ 2 3] +
(-0.5891210037060842+0j) [3^ 3] +
(0.0983952917427351+0j) [3^ 3^ 1 1] +
(0.3265353734712868+0j) [3^ 3^ 3 3]
```

Compare against PySCF [5].

In [31]:

```
mol = MolecularData(atom, basis, multiplicity=1)
mol = run_pyscf(mol)

H_ref = get_fermion_operator(mol.get_molecular_hamiltonian())
print(H_ref)
```

```
0.52917721092 [] +
-1.110844179883727 [0^ 0] +
0.3132012497647589 [0^ 0^ 0 0] +
0.09839529174273509 [0^ 0^ 2 2] +
0.3132012497647589 [0^ 1^ 1 0] +
0.09839529174273509 [0^ 1^ 3 2] +
0.09839529174273509 [0^ 2^ 0 2] +
0.3108533815598565 [0^ 2^ 2 0] +
0.09839529174273509 [0^ 3^ 1 2] +
0.3108533815598565 [0^ 3^ 3 0] +
0.3132012497647589 [1^ 0^ 0 1] +
0.09839529174273509 [1^ 0^ 2 3] +
-1.110844179883727 [1^ 1] +
0.3132012497647589 [1^ 1^ 1 1] +
0.09839529174273509 [1^ 1^ 3 3] +
0.09839529174273509 [1^ 2^ 0 3] +
0.3108533815598565 [1^ 2^ 2 1] +
0.09839529174273509 [1^ 3^ 1 3] +
0.3108533815598565 [1^ 3^ 3 1] +
0.31085338155985676 [2^ 0^ 0 2] +
0.09839529174273509 [2^ 0^ 2 0] +
0.31085338155985676 [2^ 1^ 1 2] +
0.09839529174273509 [2^ 1^ 3 0] +
-0.5891210037060843 [2^ 2] +
0.09839529174273509 [2^ 2^ 0 0] +
0.3265353734712868 [2^ 2^ 2 2] +
0.09839529174273509 [2^ 3^ 1 0] +
0.3265353734712868 [2^ 3^ 3 2] +
0.31085338155985676 [3^ 0^ 0 3] +
0.09839529174273509 [3^ 0^ 2 1] +
0.31085338155985676 [3^ 1^ 1 3] +
0.09839529174273509 [3^ 1^ 3 1] +
0.09839529174273509 [3^ 2^ 0 1] +
0.3265353734712868 [3^ 2^ 2 3] +
-0.5891210037060843 [3^ 3] +
0.09839529174273509 [3^ 3^ 1 1] +
0.3265353734712868 [3^ 3^ 3 3]
```

Good agreement.

Transitioning from electrons to qubits

To implement second-quantised electronic states and Hamiltonians on a quantum computer, they are mapped to operators acting on qubits, expressed as linear combinations of tensor products of single-qubit Pauli operators.

On qubit p, the elementary Pauli operators $\hat{I}_p, \hat{X}_p, \hat{Y}_p, \hat{Z}_p$ act on the computational basis $\{|0_p\rangle, |1_p\rangle\}$ and are represented by

$$\mathbf{I} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad \mathbf{X} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad \mathbf{Y} = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad \mathbf{Z} = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}.$$

In [32]:
```
Image('2_4.png')
```

Out[32]:

Task II.4: Data structure and algebras for qubit operators.

This task may also be done with the aid of appropriate operator classes in OpenFermion.

(a) Program a data structure entitled `PauliString` to represent the elementary Pauli operators acting on the p^{th} qubit, $\hat{I}_p$, $\hat{X}_p$, $\hat{Y}_p$, and $\hat{Z}_p$.

(b) Extend `PauliString` to handle an arbitrary string of elementary Pauli operators, *e.g.* $\hat{X}_1 \otimes \hat{I}_2 \otimes \hat{Y}_3 \otimes \hat{Z}_4$.

(c) Program another data structure entitled `QubitOperator` to handle *multiple* additive instances of `PauliString`, each of which is accompanied by a complex coefficient, *e.g.* $0.5(\hat{X}_1 \otimes \hat{Z}_2 \otimes \hat{Y}_3 \otimes \hat{I}_4) + 1.5(\hat{Z}_1 \otimes \hat{Y}_2 \otimes \hat{I}_3 \otimes \hat{X}_4)$.

(d) Implement the additive and multiplicative complex algebras for `QubitOperator` so that multiple of its instances can be added or multiplied together to give another instance. For example, with the aid of Table 1,

- $1.5(\hat{X}_1 \otimes \hat{Y}_2) + 0.5(\hat{X}_1 \otimes \hat{Y}_2) = 2.0(\hat{X}_1 \otimes \hat{Y}_2)$,
- $1.5(\hat{X}_1 \otimes \hat{Y}_2) \times 0.5(\hat{X}_1 \otimes \hat{Z}_2) = 0.75i(\hat{I}_1 \otimes \hat{X}_2)$.

First verify multiplication rules, leaving out identity since it is trivial. Do they agree with the table given in the laboratory manual [1]?

In [33]:
```python
Image('mult_table.png')
```

Out[33]:

$\times$	$\hat{I}$	$\hat{X}$	$\hat{Y}$	$\hat{Z}$
$\hat{I}$	$\hat{I}$	$\hat{X}$	$\hat{Y}$	$\hat{Z}$
$\hat{X}$	$\hat{X}$	$\hat{I}$	$i\hat{Z}$	$-i\hat{Y}$
$\hat{Y}$	$\hat{Y}$	$-i\hat{Z}$	$\hat{I}$	$i\hat{X}$
$\hat{Z}$	$\hat{Z}$	$i\hat{Y}$	$-i\hat{X}$	$\hat{I}$

In [34]:
```python
X = np.array([[0, 1], [1, 0]], dtype=complex)
Y = np.array([[0, -1j], [1j, 0]], dtype=complex)
Z = np.array([[1, 0], [0, -1]], dtype=complex)

print("XX =",X@X)
print("\nXY =",X@Y)
print("\nXZ =",X@Z)
print("\nYX =",Y@X)
print("\nYY =",Y@Y)
print("\nYZ =",Y@Z)
print("\nZX =",Z@X)
print("\nZY =",Z@Y)
print("\nZZ =",Z@Z)
```

```
XX = [[1.+0.j 0.+0.j]
 [0.+0.j 1.+0.j]]

XY = [[0.+1.j 0.+0.j]
 [0.+0.j 0.-1.j]]

XZ = [[ 0.+0.j -1.+0.j]
 [ 1.+0.j  0.+0.j]]

YX = [[0.-1.j 0.+0.j]
 [0.+0.j 0.+1.j]]

YY = [[1.+0.j 0.+0.j]
 [0.+0.j 1.+0.j]]

YZ = [[0.+0.j 0.+1.j]
 [0.+1.j 0.+0.j]]

ZX = [[ 0.+0.j  1.+0.j]
 [-1.+0.j  0.+0.j]]

ZY = [[0.+0.j 0.-1.j]
 [0.-1.j 0.+0.j]]

ZZ = [[1.+0.j 0.+0.j]
 [0.+0.j 1.+0.j]]
```

As desired.

(a)

In [35]:
```
class PauliString:
    def __init__(self, ops=()):  #ops like [(1,"X"), (2,"I"),...]
        self._qop = ofQubitOperator(())  #identity
#(b)
        for p, op in ops:
            if op != "I":
                self._qop *= ofQubitOperator(((p, op),), 1.0)

    def __str__(self):
        return str(self._qop)

    def to_openfermion(self):
        return self._qop
```

Note that an error message is printed if a string other than I, X, Y, or Z is entered.

In [36]:
```
print(PauliString([(1,"X"),(2,"I"),(3,"Y"),(4,"Z")]))
```

```
1.0 [X1 Y3 Z4]
```

(c)

In [37]:
```
class QubitOperator:
    def __init__(self, terms=()):
        self._qop = ofQubitOperator()#zero operator
        for coeff, ps in terms:
            self._qop += coeff * ps.to_openfermion()

    def to_openfermion(self):
        return self._qop

    def __str__(self):
        return str(self._qop)
#(d)
    def __add__(self, other):
        out = QubitOperator()
        out._qop = self._qop + other._qop
        return out

    def __mul__(self, other):
        out = QubitOperator()
        out._qop = self._qop * other._qop
        return out

    def __rmul__(self, other):
        out = QubitOperator()
        out._qop = other * self._qop
        return out
```

In [38]:
```
psA = PauliString([(1,"X"), (2,"Y")])
psB = PauliString([(1,"X"), (2,"Z")])
a=QubitOperator([(1.5,psA),(0.5,psA)])
b=QubitOperator([(1.5,psA)])
c=QubitOperator([(0.5,psB)])
print(a)
print(b*c)
```

```
2.0 [X1 Y2]
0.75j [X2]
```

In [39]:
```
Image('2_5.png')
```

Out[39]:

Task II.5: Occupation-number vectors as Pauli strings.

Write code that takes in an occupation-number vector $\boldsymbol{k}$ and outputs the corresponding `PauliString` containing only $\hat{X}_p$ gates that brings the qubit register from the vacuum state [Equation (3.7)] to the state $|\boldsymbol{k}\rangle$.

In [40]:
```
def occupation_to_paulistring(k):
    ops = [(p, "X") for p, occ in enumerate(k) if occ]
    return PauliString(ops)

ps = occupation_to_paulistring(k1)
print(ps)
```

```
1.0 [X0 X2]
```

Jordan-Wigner transformation

In the occupation basis, each spin-orbital p is mapped to a qubit with $|1\rangle$ denoting occupied and $|0\rangle$ denoting vacant.

To represent fermionic creation and annihilation operators by qubit operators that act identically on occupation-number states, define the Jordan-Wigner qubit operators

$$\hat{q}_p^\dagger = \left(\bigotimes_{q=0}^{p-1} \hat{Z}_q\right) \frac{\hat{X}_p - i\hat{Y}_p}{2}, \qquad \hat{q}_p = \left(\bigotimes_{q=0}^{p-1} \hat{Z}_q\right) \frac{\hat{X}_p + i\hat{Y}_p}{2}.$$

Then the mapping is

$$|\mathbf{k}\rangle \;\leftrightarrow\; |k_0 k_1 \cdots k_{M-1}\rangle, \qquad \hat{a}_p^\dagger \;\leftrightarrow\; \hat{q}_p^\dagger, \qquad \hat{a}_p \;\leftrightarrow\; \hat{q}_p,$$

so fermionic operators (and hence $\hat{H}$) can be converted into sums of Pauli strings acting on qubits [1].

Verifying that $\hat{a}_p^\dagger \leftrightarrow \hat{q}_p^\dagger$ and $\hat{a}_p \leftrightarrow \hat{q}_p$. Noting that for this example the first q has index 1, like in the manual.

In [41]:
```
Image('JW.jpg')
```

Out[41]:

$$|1\rangle = \begin{pmatrix}1\\0\end{pmatrix},\ |0\rangle = \begin{pmatrix}0\\1\end{pmatrix},\ \hat{X}_p = \begin{pmatrix}0&1\\1&0\end{pmatrix},\ \hat{Y}_p = \begin{pmatrix}0&-i\\i&0\end{pmatrix}$$

$$\frac{1}{2}(\hat{X}_p - iY_p) = \begin{pmatrix}0&0\\1&0\end{pmatrix},\ \frac{1}{2}(\hat{X}_p + iY_p) = \begin{pmatrix}0&1\\0&0\end{pmatrix}$$

$$|1\rangle\langle 0| = \begin{pmatrix}0&0\\1&0\end{pmatrix} = \sigma^+,\ |0\rangle\langle 1| = \begin{pmatrix}0&1\\0&0\end{pmatrix} = \sigma^- \Rightarrow \frac{1}{2}(\hat{X}_p - iY_p) = \sigma_p^+,\ \frac{1}{2}(\hat{X}_p + iY_p) = \sigma_p^-$$

$$\sigma^+|0\rangle = |1\rangle\langle 0|0\rangle = |1\rangle,\ \sigma^+|1\rangle = |1\rangle\langle 0|1\rangle = 0 \Rightarrow \delta_{k_p 0} \text{ enforced}$$

$$\sigma^-|0\rangle = |0\rangle\langle 1|0\rangle = 0,\ \sigma^-|1\rangle = |0\rangle\langle 1|1\rangle = |0\rangle \Rightarrow \delta_{k_p 1} \text{ enforced}$$

$$\hat{Z}_q |k_q\rangle = (-1)^{k_q}|k_q\rangle$$

$$\Rightarrow \left(\bigotimes_{q=1}^{p-1} Z_q\right)|\underline{k}\rangle = (-1)^{\sum_{q<p} k_q}|\underline{k}\rangle = \prod_{q=1}^{p-1}(-1)^{k_q}|\underline{k}\rangle = \Gamma_p^{\underline{k}}|\underline{k}\rangle$$

$$\Rightarrow \hat{a}_p^\dagger \leftrightarrow \hat{q}_p^\dagger,\ \hat{a}_p \leftrightarrow \hat{q}_p$$

In [42]:
```
Image('2_6a.png')
```

Out[42]:

Task II.6: Jordan–Wigner qubit Hamiltonian for the H_2 molecule.

This task may also be done with the aid of appropriate transformation functions in OpenFermion.

(a) Write code to convert an instance of `FermionicOperator` (Task I.3) to an

In [43]:
```
Image('2_6b.png')
```

Out[43]:

instance of `QubitOperator` (Task II.4) using the Jordan–Wigner mapping in Equation (4.2).

(b) Use this code to obtain the Jordan–Wigner qubit electronic Hamiltonian from the second-quantised electronic Hamiltonian for the H_2 molecule calculated in Task II.3(d).

(a)

In [44]:
```
# rough work for of_ferm.terms.items()
a=FermionicOperator.string([(1,1), (2,1), (3,0), (4,0)], 0.5).to_openfermion()
a.terms.items()
```

Out[44]:
```
dict_items([(((1, 1), (2, 1), (3, 0), (4, 0)), (0.5+0j))])
```

In [45]:
```
def jw_transform(ferm_op):
    of_ferm = ferm_op.to_openfermion()
    qop = ofQubitOperator()

    for term, coeff in of_ferm.terms.items():
        jw = ofQubitOperator((), 1.0)

        for p, ca in term:
            # Z string
            for q in range(p):
                jw *= ofQubitOperator(((q,"Z"),), 1.0)
            if ca == 1:
                #creation
                jw *= (0.5 * ofQubitOperator(((p,"X"),), 1.0) - 0.5j * ofQubitOperator(((p,"Y"),), 1.0))
            else:
```

```
            # annihilation
            jw *= (0.5 * ofQubitOperator(((p,"X"),), 1.0) + 0.5j * ofQubitOperator(((p,"Y"),), 1.0))

        qop += complex(coeff) * jw

    mapped = QubitOperator()
    mapped._qop = qop
    #map to QubitOperator class
    return mapped
```

(b)

In [46]:
```
H_q = jw_transform(H)
print(H_q)
```

```
(-0.32760818967481076+0j) [] +
(-0.049197645871367546+0j) [X0 X1 Y2 Y3] +
(0.049197645871367546+0j) [X0 Y1 Y2 X3] +
(0.049197645871367546+0j) [Y0 X1 X2 Y3] +
(-0.049197645871367546+0j) [Y0 Y1 X2 X3] +
(0.1371657293709492+0j) [Z0] +
(0.15660062488237944+0j) [Z0 Z1] +
(0.10622904490856078+0j) [Z0 Z2] +
(0.15542669077992832+0j) [Z0 Z3] +
(0.13716572937099483+0j) [Z1] +
(0.15542669077992832+0j) [Z1 Z2] +
(0.10622904490856078+0j) [Z1 Z3] +
(-0.13036292057109045+0j) [Z2] +
(0.1632676867356434+0j) [Z2 Z3] +
(-0.13036292057109042+0j) [Z3]
```

Note that Pauli operators acting on different qubits commute (i.e. $\hat{X}_0\hat{X}_1\hat{Y}_2\hat{Y}_3 = \hat{Y}_2\hat{X}_1\hat{X}_0\hat{Y}_3$), so order doesn't matter. Pauli operators acting on the same qubit generally do not commute (i.e. $\hat{X}_p\hat{Y}_p = i\hat{Z}_p$ while $\hat{Y}_p\hat{X}_p = -i\hat{Z}_p$), so ordering matters when multiple Paulis act on the same qubit.

Compare against PySCF.

In [47]:
```
atom = """
H 0 0 0
H 0 0 1
"""
mol = MolecularData(atom, "sto-3g", multiplicity=1)
mol = run_pyscf(mol)

H_ref = jordan_wigner(mol.get_molecular_hamiltonian())
print(H_ref)
```

```
-0.3276081896748101 [] +
-0.049197645871367546 [X0 X1 Y2 Y3] +
0.049197645871367546 [X0 Y1 Y2 X3] +
0.049197645871367546 [Y0 X1 X2 Y3] +
-0.049197645871367546 [Y0 Y1 X2 X3] +
0.13716572937099492 [Z0] +
0.15660062488237944 [Z0 Z1] +
0.10622904490856078 [Z0 Z2] +
0.15542669077992832 [Z0 Z3] +
0.13716572937099492 [Z1] +
0.15542669077992832 [Z1 Z2] +
0.10622904490856078 [Z1 Z3] +
-0.13036292057109033 [Z2] +
0.1632676867356434 [Z2 Z3] +
-0.13036292057109033 [Z3]
```

In [48]:
```
Image('2_7.png')
```

Out[48]:

Task II.7: Jordan–Wigner Slater determinants.

This task is optional.

(a) Use the code written in Task II.6(a) to convert the second-quantised form of Slater determinants constructed in Task II.1 to the Jordan–Wigner qubit form.

(b) Essentially, both the qubit operator constructed above and the Pauli string constructed in Task II.5 transform the vacuum state to $|\boldsymbol{k}\rangle$. However, they are not identical and only the qubit operator constructed above is the faithful map of the creation operator string in Equation (3.11). Can you explain the differences between them?

(a)

In [49]:
```python
op = occupation_to_operator(k1)
print('For second-quantized k1 =',op.to_openfermion(),'the Jordan-Wigner qubit form is:',jw_transform(op))
```

```
For second-quantized k1 = (1+0j) [0^ 2^] the Jordan-Wigner qubit form is: (0.25+0j) [X0 Z1 X2] +
-0.25j [X0 Z1 Y2] +
-0.25j [Y0 Z1 X2] +
(-0.25+0j) [Y0 Z1 Y2]
```

In [50]:
```python
Image('JWexample.jpg')
```

Out[50]:

J-W mapping of $\hat{a}_0^\dagger \hat{a}_2^\dagger$

$$\hat{a}_0^\dagger = \frac{X_0 - iY_0}{2}, \quad \hat{a}_2^\dagger = Z_0 Z_1 \frac{X_2 - iY_2}{2}$$

$$\Rightarrow \hat{a}_0^\dagger \hat{a}_2^\dagger = \frac{1}{4} \underbrace{(X_0 - i\overbrace{Y_0) Z_0}^{iX_0}}_{-iY_0} Z_1 (X_2 - iY_2) = \frac{1}{4}(X_0 - iY_0) Z_1 (X_2 - iY_2)$$

$$\Rightarrow \hat{a}_0^\dagger \hat{a}_2^\dagger = \frac{1}{4}(X_0 Z_1 X_2 - iX_0 Z_1 Y_2 - iY_0 Z_1 X_2 - Y_0 Z_1 Y_2)$$

(b)

Jordan-Wigner mapping is the true operator map of $\prod_p (a_p^\dagger)^{k_p}$, including the parity string $\bigotimes_{q=0}^{p-1} \hat{Z}_q$ and the $(\hat{X}_p \mp i\hat{Y}_p)/2$ structure, so it expands into a sum of Pauli strings that preserves fermionic anticommutation.

The Pauli string $\hat{X}_0 \hat{X}_2$ is a state preparation shortcut that flips $|000\rangle \to |101\rangle$. It does not enforce fermionic occupancy rules or anticommutation. For example $\hat{X}_0 \hat{X}_2 |011\rangle = |110\rangle$, while $\hat{a}_0^\dagger \hat{a}_2^\dagger |011\rangle = 0$ because creating on an already occupied mode gives zero.

Module 2 Logbook

This laboratory extended the second quantisation formalism to the electronic Hamiltonian and introduced the Jordan-Wigner transformation, which maps fermionic creation and annihilation operators to Pauli operators on qubits. A comparison between simply applying X operators and the full Jordan-Wigner mapping showed that while the simple method can reproduce the correct bitstring for some state preparations, it does not implement fermionic creation and annihilation operators.

The Jordan-Wigner mapping was implemented with the parity Z string and the $(X \mp iY)/2$ structure, applied term by term to each fermionic string. Even though each elementary fermionic operator maps to only two Pauli terms, multiplying several of them together produces large strings.

The main computational consideration for PauliString and QubitOperator was to keep them as simple and general as possible. OpenFermion's QubitOperator was used for the bookkeeping, combining like terms and keeping a consistent ordering. A thin wrapper was then added to provide addition and multiplication.

Correctness was checked using the same workflow the code is meant for. The elementary Pauli multiplication rules were verified directly by matrix multiplication of X, Y, and Z. Then, the PauliString and QubitOperator classes were tested on the specific addition and multiplication examples provided, confirming that identical Pauli strings combined correctly by coefficient addition and that multiplication produced the expected phase factors and Pauli products. The second quantised H_2 Hamiltonian constructed from the UHF Molecular Orbital transformation was compared against an OpenFermion reference Hamiltonian generated from PySCF output, and the term list agreed (up to numerical precision). Finally, the Jordan-Wigner Hamiltonian produced by the custom mapping was compared against OpenFermion's built in transform, and the resulting Pauli terms and coefficients matched.

Part III

Simulation Framework

Classical simulation of state-vectors

An M-qubit pure state in the computational basis can be stored classically as a length 2^M state-vector of complex amplitudes. For a product state

$$|\Psi_q\rangle = \bigotimes_{p=0}^{M-1} |\psi_p\rangle, \qquad |\psi_p\rangle = c_{p,0}|0_p\rangle + c_{p,1}|1_p\rangle, \quad |c_{p,0}|^2 + |c_{p,1}|^2 = 1,$$

the corresponding state-vector entries are amplitudes for basis states $|i_0 i_1 \cdots i_{M-1}\rangle$:

$$V_{i_0 i_1 \cdots i_{M-1}} = \prod_{p=0}^{M-1} c_{p,i_p}, \qquad i_p \in \{0,1\},$$

and $|V_{i_0 i_1 \cdots i_{M-1}}|^2$ gives the associated frequency [1].

In [51]:
```
Image('3_1a.png')
```

Out[51]:

Task III.1: Classical state-vector simulations with Pauli operators.

You may find NumPy helpful for this task.

(a) Write code to take in a simple string of M bits (*e.g.* '0000' or '1011') representing

In [52]:
```
Image('3_1b.png')
```

Out[52]:

a computational basis state and output the corresponding state-vector $\boldsymbol{V}$ as an M-dimensional array.

(b) By making use of the representation matrices in Equation (4.1), and by using Equation (5.4), extend the above code to obtain the transformed state-vector $\boldsymbol{V}'$ under the action of *a single string of elementary Pauli operators* (*e.g.* $X_1 \otimes I_2 \otimes Y_3 \otimes Z_4$) represented by the `PauliString` data structure [Task II.4(b)].

(c) From $\boldsymbol{V}'$, compute the frequencies of the computational basis states.

(a)

In [53]:
```
def b_svtensor(bstr):
    bits = tuple(int(b) for b in bstr.strip())
    M = len(bits)
    V = np.zeros((2,) * M, dtype=complex)
    V[bits] = 1 #sets amplitude of bstr to 1
    return V
```

In [54]:
```
V = b_svtensor("1011")
print("Shape of V:",V.shape)
print("V[1,0,1,1]:",V[1,0,1,1])
print("V:",V)
```

```
Shape of V: (2, 2, 2, 2)
V[1,0,1,1]: (1+0j)
V: [[[[0.+0.j 0.+0.j]
   [0.+0.j 0.+0.j]]

  [[0.+0.j 0.+0.j]
   [0.+0.j 0.+0.j]]]


 [[[0.+0.j 0.+0.j]
   [0.+0.j 1.+0.j]]

  [[0.+0.j 0.+0.j]
   [0.+0.j 0.+0.j]]]]
```

(b)

Note 3

If a k-qubit unitary $\hat{U}$ acts on selected qubits, the updated state-vector V' is obtained by contracting $\hat{U}$ with V over the acted-upon indices. $\Rightarrow V'_{\cdots i'_{p-1} a i'_{p+1} \cdots} = \sum_b V_{\cdots i_{p-1} b i_{p+1} \cdots} U_{ab}$.

Since the state-vector dimension scales as 2^M, only small M classical simulations are possible [1].

In [55]:
```
pauli = {"I": np.eye(2, dtype=complex),"X": np.array([[0, 1], [1, 0]], dtype=complex),
         "Y": np.array([[0, -1j], [1j, 0]], dtype=complex),"Z": np.array([[1, 0], [0, -1]], dtype=complex)}

def apply_paulistring(V, ps):
    #V from b_svtensor
    qop = ps.to_openfermion()
    terms = list(qop.terms.items())

    term, coeff = terms[0]
    # term is e.g. ((p,'X'), (q,'Z'), ...)
    Vp = V * coeff
    for p, op in term:
        Vp = np.tensordot(pauli[op], Vp, axes=([1], [p])) #note 3
        Vp = np.moveaxis(Vp, 0, p) #tensordot inserts the new output index of the Pauli matrix as axis 0
    return Vp
```

(c)

In [56]:
```
def frequencies(Vp):
    probs = np.abs(Vp)**2
    return probs
```

In [57]:
```
V = b_svtensor("1011")
ps = PauliString([(0,"X"), (1,"I"), (2,"Y"), (3,"Z")])
#arbitrary

Vp = apply_paulistring(V, ps)
freq = frequencies(Vp)
print("V':",Vp)
print("\nFrequency:",freq)
print("\nSum of frequencies:",freq.sum())
print("V'[0,0,0,1]:",Vp[0,0,0,1])
```

```
V': [[[[0.+0.j 0.+1.j]
   [0.+0.j 0.+0.j]]

  [[0.+0.j 0.+0.j]
   [0.+0.j 0.+0.j]]]


 [[[0.+0.j 0.+0.j]
   [0.+0.j 0.+0.j]]

  [[0.+0.j 0.+0.j]
   [0.+0.j 0.+0.j]]]]

Frequency: [[[[0. 1.]
   [0. 0.]]

  [[0. 0.]
   [0. 0.]]]


 [[[0. 0.]
   [0. 0.]]

  [[0. 0.]
   [0. 0.]]]]

Sum of frequencies: 1.0
V'[0,0,0,1]: 1j
```

Try $X_0 Z_0 \otimes I_1 \otimes Y_2 \otimes Z_3 \otimes Z_4$.

In [58]:
```
V = b_svtensor("10110")

ps = PauliString([(4,"Z")])
Vp = apply_paulistring(V, ps)

ps1 = PauliString([(4,"X"), (3,"I"), (2,"Y"), (1,"Z"),(0,"Z")])
Vp = apply_paulistring(Vp, ps1)
freq = frequencies(Vp)

print("V':",Vp)
print("\nFrequency:",freq)
print("\nSum of frequencies:",freq.sum())
print("V'[1,0,0,1,1]:",Vp[1,0,0,1,1])
```

```
V': [[[[[0.+0.j 0.+0.j]
    [0.+0.j 0.+0.j]]

   [[0.+0.j 0.+0.j]
    [0.+0.j 0.+0.j]]]


  [[[0.+0.j 0.+0.j]
    [0.+0.j 0.+0.j]]

   [[0.+0.j 0.+0.j]
    [0.+0.j 0.+0.j]]]]



 [[[[0.+0.j 0.+0.j]
    [0.+0.j 0.+1.j]]

   [[0.+0.j 0.+0.j]
    [0.+0.j 0.+0.j]]]


  [[[0.+0.j 0.+0.j]
    [0.+0.j 0.+0.j]]

   [[0.+0.j 0.+0.j]
    [0.+0.j 0.+0.j]]]]]
Frequency: [[[[[0. 0.]
    [0. 0.]]

   [[0. 0.]
    [0. 0.]]]


  [[[0. 0.]
    [0. 0.]]

   [[0. 0.]
    [0. 0.]]]]



 [[[[0. 0.]
    [0. 1.]]

   [[0. 0.]
    [0. 0.]]]


  [[[0. 0.]
    [0. 0.]]

   [[0. 0.]
    [0. 0.]]]]]
Sum of frequencies: 1.0
V'[1,0,0,1,1]: 1j
```

Now compare against Qiskit's classical state-vector simulators. Note that Qiskit uses the rightmost bit as qubit 0 convention, which will be used for the remaining sections.

In [59]:
```
Image('3_2.png')
```

Out[59]:

Task III.2: Classical state-vector simulations using a library.

You will need to consult the relevant API documentations for your quantum-computing library of choice such as Qiskit to complete this task.

(a) Use a high-level quantum-computing library to construct a quantum circuit of M qubits and initialise it to a computational basis state represented by a simple string of M bits (*e.g.* ‘0000’ or ‘1011’).

(b) Append a layer of Pauli gates to the above circuit based on a string of elementary Pauli operators represented by the `PauliString` data structure [Task II.4(b)].

(c) Simulate the circuit to obtain the state-vector representing the outcome and compute the frequencies of the computational basis states.

In [60]:
```
service = QiskitRuntimeService(name="oxford-chem")
```

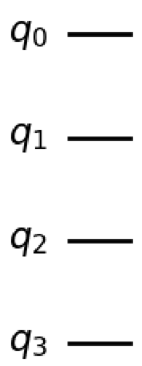
```
management.get:WARNING:2026-03-09 10:30:35,457: Loading saved account: oxford-chem
```

(a)

In [61]:
```
qc0000 = QuantumCircuit(4)


print("'0000'")
qc0000.draw("mpl")
```

```
'0000'
```

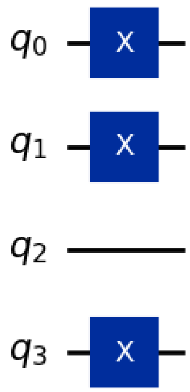
Out[61]:



In [62]:
```
qc1011 = QuantumCircuit(4)
qc1011.x(0)
qc1011.x(1)
qc1011.x(3)
print("'1011'")
qc1011.draw("mpl")
```

```
'1011'
```

Out[62]:

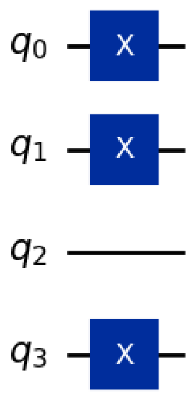


(b)

In [63]:
```python
def append_pauli(qc, ps):
    term = next(iter(ps.to_openfermion().terms)) #next gets first key from dict
    for p, op in term:
        if op == "X":
            qc.x(p)
        elif op == "Y":
            qc.y(p)
        elif op == "Z":
            qc.z(p)
    return qc
```

In [64]:
```python
qc = QuantumCircuit(4)

ps = PauliString([(0, "X"), (1, "X"), (3, "X")])

append_pauli(qc, ps)
qc.draw("mpl")
```

Out[64]:



(c)

Check agreement with III.1 (a)

In [65]:
```python
sv = Statevector.from_instruction(qc)
print("Statevector :", sv.data)
probs = sv.probabilities_dict()
print("Probabilities :", probs)
```

```
Statevector : [0.+0.j 0.+0.j 0.+0.j 0.+0.j 0.+0.j 0.+0.j 0.+0.j 0.+0.j 0.+0.j 0.+0.j
 0.+0.j 1.+0.j 0.+0.j 0.+0.j 0.+0.j 0.+0.j]
Probabilities : {np.str_('1011'): np.float64(1.0)}
```

Statevector agrees with III.1 (a)

Comments are from IBM Quantum learning [6].

In [66]:
```python
backend = AerSimulator()

print(backend.name)
```

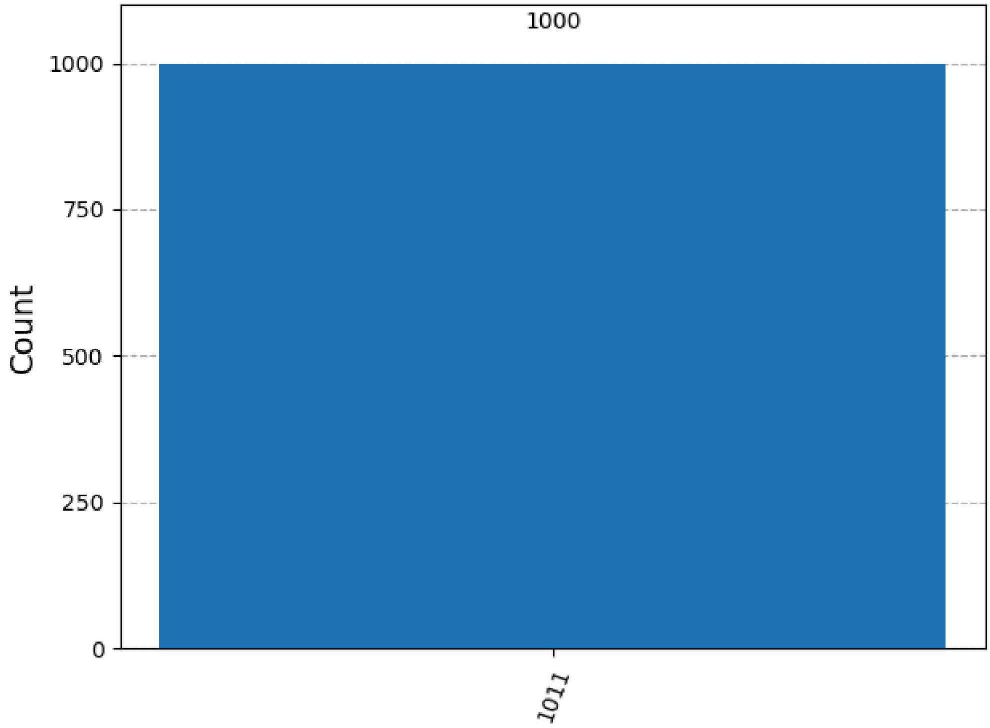
```
aer_simulator
```

In [67]:
```
## Transpile
target = backend.target
pm = generate_preset_pass_manager(target=target, optimization_level=3)
qc_1=qc.copy()
qc_1.measure_all()
qc_isa = pm.run(qc_1)
```

In [68]:
```
sampler = Sampler(mode=backend)
pubs = [qc_isa] # Primitive Unified Blocs, holds circuit
job = sampler.run(pubs, shots=1000) # repeats the circuit 1000 times
res = job.result()

counts = res[0].data.meas.get_counts()
```

In [69]:
```
## Analysis

plot_histogram(counts)
#plt.hist(counts)
```

Out[69]:

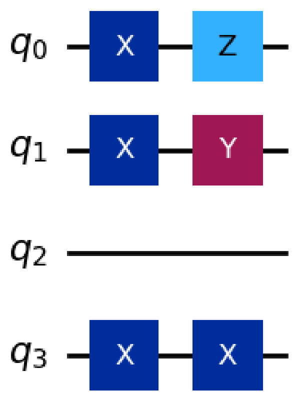


Aer simulator gives expected result for III.1 (a)

Check agreement with III.1(c)

In [70]:
```
ps1 = PauliString([(3,"X"), (2,"I"), (1,"Y"), (0,"Z")])
append_pauli(qc, ps1)
qc.draw("mpl")
```

Out[70]:



In [71]:
```
sv = Statevector.from_instruction(qc)
print("Statevector :", sv.data)
probs = sv.probabilities_dict()
print("Probabilities :", probs)
```

```
Statevector : [0.+0.j 0.+1.j 0.+0.j 0.+0.j 0.+0.j 0.+0.j 0.+0.j 0.+0.j 0.+0.j 0.+0.j
 0.+0.j 0.+0.j 0.+0.j 0.+0.j 0.+0.j 0.+0.j]
Probabilities : {np.str_('0001'): np.float64(1.0)}
```

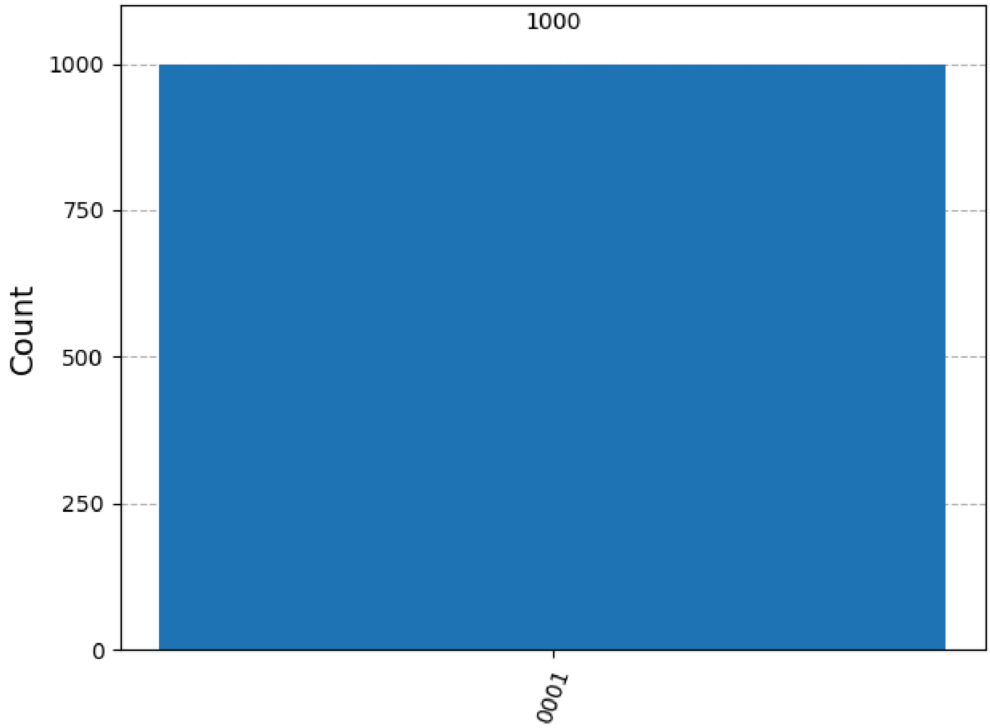
Statevector agrees with III.1 (c)

In [72]:
```
## Transpile
target = backend.target
pm = generate_preset_pass_manager(target=target, optimization_level=3)
qc_2=qc.copy()
qc_2.measure_all()
qc_isa = pm.run(qc_2)
```

In [73]:
```
sampler = Sampler(mode=backend)
pubs = [qc_isa] # Primitive Unified Blocs, holds circuit
job = sampler.run(pubs, shots=1000) # repeats the circuit 1000 times
res = job.result()

counts = res[0].data.meas.get_counts()
```

In [74]:
```
## Analysis

plot_histogram(counts)
#plt.hist(counts)
```

Out[74]:

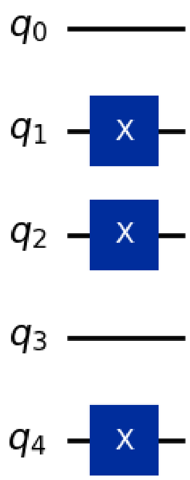


Aer simulator gives expected result for III.1 (c)

Check agreement with $X_0 Z_0 \otimes I_1 \otimes Y_2 \otimes Z_3 \otimes Z_4$.

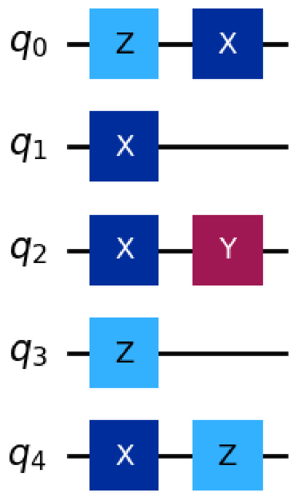
In [75]:
```
qc = QuantumCircuit(5)

ps = PauliString([(1, "X"), (2, "X"), (4, "X")])

append_pauli(qc, ps)

qc.draw("mpl")
```

Out[75]:



In [76]:
```
ps2 = PauliString([(0,"Z")])
append_pauli(qc, ps2)
ps3 = PauliString([(0,"X"),(4,"Z"), (3,"Z"), (2,"Y"), (1,"I")]) #in little endian
append_pauli(qc, ps3)
qc.draw("mpl")
```

Out[76]:



In [77]:
```
sv = Statevector.from_instruction(qc)
print("Statevector :", sv.data)
probs = sv.probabilities_dict()
print("Probabilities :", probs)
```

```
Statevector : [0.+0.j 0.+0.j 0.+0.j 0.+0.j 0.+0.j 0.+0.j 0.+0.j 0.+0.j 0.+0.j 0.+0.j
 0.+0.j 0.+0.j 0.+0.j 0.+0.j 0.+0.j 0.+0.j 0.+0.j 0.+0.j 0.+0.j 0.+1.j
 0.+0.j 0.+0.j 0.+0.j 0.+0.j 0.+0.j 0.+0.j 0.+0.j 0.+0.j 0.+0.j 0.+0.j
 0.+0.j 0.+0.j]
Probabilities : {np.str_('10011'): np.float64(1.0)}
```

Statevector agrees.

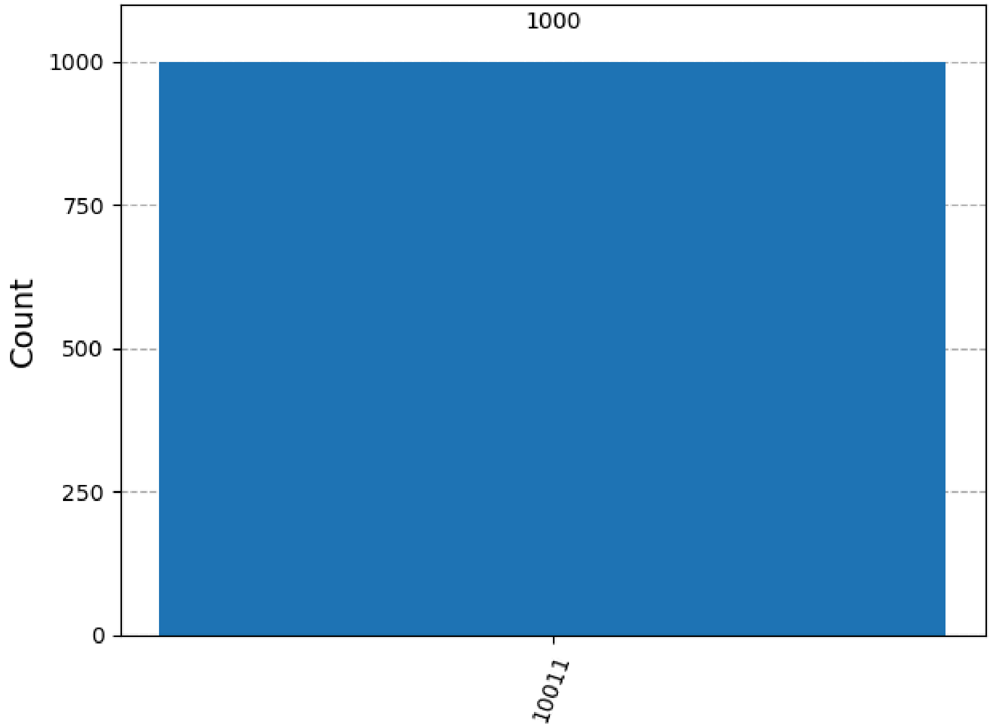
In [78]:
```
## Transpile
target = backend.target
pm = generate_preset_pass_manager(target=target, optimization_level=3)
qc_2=qc.copy()
qc_2.measure_all()
qc_isa = pm.run(qc_2)
```

In [79]:
```
sampler = Sampler(mode=backend)
pubs = [qc_isa] # Primitive Unified Blocs, holds circuit
job = sampler.run(pubs, shots=1000) # repeats the circuit 1000 times
res = job.result()

counts = res[0].data.meas.get_counts()
```

In [80]:
```
## Analysis
```

```
plot_histogram(counts)
#plt.hist(counts)
```

Out[80]:



Aer simulator gives expected result.

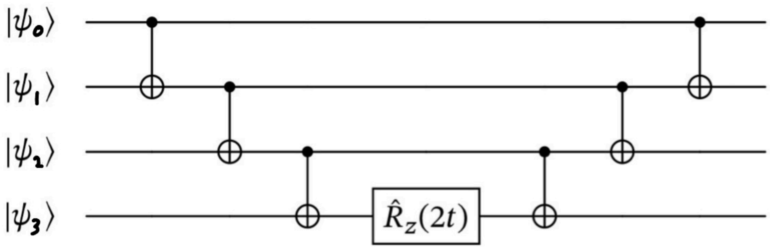
Unitary time evolution of Pauli operators

A single Pauli string $\hat{P}$ is both Hermitian and unitary, so its time-evolution operator $e^{-it\hat{P}}$ is unitary and can be implemented as a circuit.

For $\hat{P} = \hat{Z}_1 \otimes \hat{Z}_2 \otimes \cdots \otimes \hat{Z}_M$, the unitary $e^{-it\hat{P}}$ can be implemented by computing the parity of the M qubits with a CNOT ladder, applying a single $R_z(2t)$ rotation to one qubit, and then uncomputing the parity [1]. This is verified mathematically below.

In [81]:
```
Image('3_3.jpg')
```

Out[81]:



$$CNOT_{0,1}\, CNOT_{1,2}\, CNOT_{2,3}\, \hat{R}_z(2t)_3\, CNOT_{2,3}\, CNOT_{1,2}\, CNOT_{0,1}\, |dcba\rangle$$

$$= CNOT_{0,1}\, CNOT_{1,2}\, CNOT_{2,3}\, \hat{R}_z(2t)_3\, |d\oplus c\oplus b\oplus a\rangle \otimes |c\oplus b\oplus a\rangle \otimes |b\oplus a\rangle \otimes |a\rangle$$

$$= \begin{cases} CNOT_{0,1}\, CNOT_{1,2}\, CNOT_{2,3}\, e^{-it}\, |d\oplus c\oplus b\oplus a\rangle \otimes |c\oplus b\oplus a\rangle \otimes |b\oplus a\rangle \otimes |a\rangle, & d\oplus c\oplus b\oplus a = 0 \\ CNOT_{0,1}\, CNOT_{1,2}\, CNOT_{2,3}\, e^{it}\, |d\oplus c\oplus b\oplus a\rangle \otimes |c\oplus b\oplus a\rangle \otimes |b\oplus a\rangle \otimes |a\rangle, & d\oplus c\oplus b\oplus a = 1 \end{cases}$$

$$= CNOT_{0,1}\, CNOT_{1,2}\, CNOT_{2,3}\, e^{-it(-1)^{d\oplus c\oplus b\oplus a}}\, |d\oplus c\oplus b\oplus a\rangle \otimes |c\oplus b\oplus a\rangle \otimes |b\oplus a\rangle \otimes |a\rangle$$

$$= e^{-it(-1)^{d\oplus c\oplus b\oplus a}}\, |dcba\rangle$$

If $\hat{P}$ contains $\hat{X}_p$ or $\hat{Y}_p$ factors, each is first conjugated into $\hat{Z}_p$ using single-qubit basis changes,

$$\hat{X}_p = \hat{H}_p \hat{Z}_p \hat{H}_p, \qquad \hat{Y}_p = (\hat{S}_p \hat{H}_p^\dagger) \hat{Z}_p (\hat{H}_p \hat{S}_p^\dagger),$$

after which the same parity - $R_z(2t)$ - uncompute pattern applies [1].

In [82]:
```python
Image('3_3a.png')
```

Out[82]:

Task III.3: Unitary time evolution of Pauli strings.

You will need to consult the relevant API documentations for your quantum-computing library of choice such as Qiskit to complete this task.

In [83]:
```python
Image('3_3b.png')
```

Out[83]:

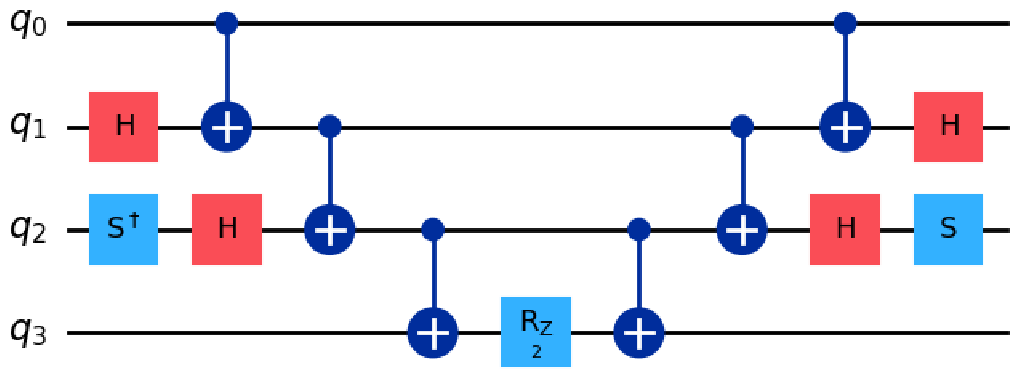
Using a high-level quantum-computing library, write code to construct the circuit for $\exp(-it\hat{P})$ for a real parameter t and a string of elementary Pauli operators $\hat{P}$ represented by the `PauliString` data structure [Task II.4(b)].

In [84]:
```python
def exp_it(ps, t, num_qubits):
    qop = ps.to_openfermion()
    terms = list(qop.terms.items())

    term, coeff = terms[0]
    qc = QuantumCircuit(num_qubits)

    basis_changes = []

    #involved qubits (non identity)
    involved = [p for p, n in sorted(term, key=lambda x: x[0])]

    #keep to uncompute
    for p, op in sorted(term, key=lambda x: x[0]):
        #sort by qubit index
        if op == "X":
            qc.h(p)
            basis_changes.append((p, "X"))
        elif op == "Y":
            qc.sdg(p) #notes have conjugation order incorrect
            qc.h(p)
            basis_changes.append((p, "Y"))
        elif op == "Z":
            basis_changes.append((p, "Z"))

    # cnot over involved qubits
    for i in range(len(involved) - 1):
        qc.cx(involved[i], involved[i+1])

    # Phase rotation
    qc.rz(2.0 * t, involved[-1])

    # uncompute
    for i in reversed(range(len(involved) - 1)):
        qc.cx(involved[i], involved[i+1])

    #undo basis changes
    for p, op in reversed(basis_changes):
        if op == "X":
            qc.h(p)
        elif op == "Y":
            qc.h(p)
            qc.s(p)
        elif op == "Z":
            pass

    return qc
```

Check if the circuit for $\exp[-it(\hat{Z}_1 \otimes \hat{X}_2 \otimes \hat{Y}_3 \otimes \hat{Z}_4)]$ is correct.

In [85]:
```python
ps = PauliString([(0,"Z"),(1,"X"), (2,"Y"), (3,"Z")])
t = 1
qc = exp_it(ps, t, num_qubits=4)
qc.draw("mpl")
```

Out[85]:

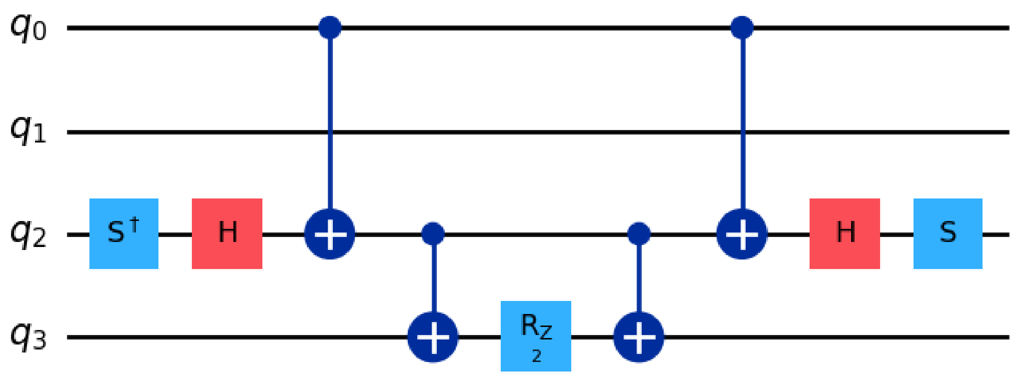


Circuit as expected.

See if the Identity is correctly accounted for, with $\exp[-it(\hat{Z}_1 \otimes \hat{I}_2 \otimes \hat{Y}_3 \otimes \hat{Z}_4)]$.

In [86]:
```
ps1 = PauliString([(0,"Z"),(1,"I"), (2,"Y"), (3,"Z")])
t = 1
qc = exp_it(ps1, t, num_qubits=4)
qc.draw("mpl")
```

Out[86]:



Answer as expected, without CNOT on qubit 1.

Trotterisation

A general qubit operator is a sum of K Pauli strings,

$$\hat{Q} = \sum_{k=0}^{K-1} c_k \hat{P}_k,$$

which is Hermitian but not unitary. Quantum simulation requires the unitary time evolution $e^{-it\hat{Q}}$, but since the Pauli string terms $\hat{P}_k$ do not necessarily commute,

$$e^{-it\sum_k c_k \hat{P}_k} \neq \prod_k e^{-itc_k\hat{P}_k}.$$

The Trotter-Suzuki formulas approximate $e^{-it\hat{Q}}$ as follows.

First order with Trotter number r:

$$e^{-it\hat{Q}} = \left[\prod_{k=0}^{K-1} \exp\left(-i\frac{t}{r}c_k\hat{P}_k\right)\right]^r + \mathcal{O}(t^2/r).$$

Second order Trotter-Suzuki:

$$e^{-it\hat{Q}} = \left[\prod_{k=0}^{K-1} \exp\left(-i\frac{t}{2r}c_k\hat{P}_k\right) \prod_{k=0}^{K-1} \exp\left(-i\frac{t}{2r}c_{K-k-1}\hat{P}_{K-k-1}\right)\right]^r + \mathcal{O}(t^3/r^2).$$

Increasing r reduces the approximation error but increases circuit depth [1].

The indexing for the second order Trotter-Suzuki formula is verified below, noting that for this example the first q has index 1, like in the manual.

In [87]:
```
Image('3_4.jpg')
```

Out[87]:

$$\exp(-it\hat{Q}) = \left[\prod_{k=1}^{K} \exp\left(-i\frac{t}{2r}c_k\hat{P}_k\right) \prod_{k=1}^{K} \exp\left(-i\frac{t}{2r}c_{K-k+1}\hat{P}_{K-k+1}\right)\right]^r + O(t^3/r^2)$$

for $\hat{Q} = c_1\hat{P}_1 + c_2\hat{P}_2$

$$\Rightarrow \exp(-it(c_1\hat{P}_1 + c_2\hat{P}_2)) = \left(\exp(-i\tfrac{t}{2r}c_1\hat{P}_1)\exp(-i\tfrac{t}{2r}c_2\hat{P}_2)\exp(-i\tfrac{t}{2r}c_2\hat{P}_2)\exp(-i\tfrac{t}{2r}c_1\hat{P}_1)\right)^r$$

$$= \left(\exp(-i\tfrac{t}{2r}c_1\hat{P}_1)\exp(-i\tfrac{t}{r}c_2\hat{P}_2)\exp(-i\tfrac{t}{2r}c_1\hat{P}_1)\right)^r$$

In [88]:
```
Image('3_4.png')
```

Out[88]:

Task III.4: Trotterised time evolution of qubit Hamiltonian for H_2.

You will need to consult the relevant API documentations for your quantum-computing library of choice such as Qiskit to complete this task.

(a) Using a high-level quantum-computing library and the code in Task III.3, write code to construct a quantum circuit for $\exp(-it\hat{Q})$ using both the first- and second-order Trotter–Suzuki decompositions with Trotter number r [Equation (5.13)], where t is a real parameter and $\hat{Q}$ a qubit operator represented by the `QubitOperator` data structure [Task II.4(d)].

(b) Use the above code to construct a quantum circuit for the Trotterised time-evolved qubit Hamiltonian of the H_2 molecule obtained from Task II.6.

(a)

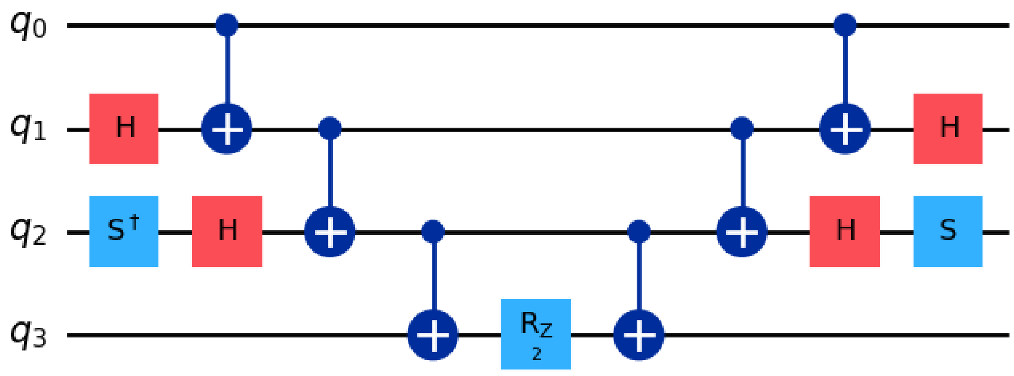
In [89]:
```
def trotter_first_order(Q, t, r, num_qubits):
    qc = QuantumCircuit(num_qubits)

    terms = []
    for term, coeff in Q.to_openfermion().terms.items():
```

```
        ps = PauliString(list(term))
        terms.append((coeff, ps))
    terms.sort(key=lambda x: str(x[1])) #deterministic term order


    dt = t / r
    for _ in range(r):
        for ck, Pk in terms:
            qc.compose(exp_it(Pk, (dt * ck).real, num_qubits), inplace=True)
            # modify rather than new circuit
    return qc

def trotter_second_order(Q, t, r, num_qubits):
    qc = QuantumCircuit(num_qubits)
    terms = []
    for term, coeff in Q.to_openfermion().terms.items():
        ps = PauliString(list(term))
        terms.append((coeff, ps))

    terms.sort(key=lambda x: str(x[1]))
    dt = t / r

    for _ in range(r):
        for ck, Pk in terms:
            qc.compose(exp_it(Pk, (dt * ck/2).real, num_qubits), inplace=True)

        for ck, Pk in reversed(terms):
            qc.compose(exp_it(Pk, (dt * ck/2).real, num_qubits), inplace=True)

    return qc
```

Confirm whether the circuit from III.3 is correctly reproduced.

In [90]:
```
ps = PauliString([(0,"Z"),(1,"X"), (2,"Y"), (3,"Z")])
t = 1
qc=trotter_first_order(ps,t,1,4)
qc.draw("mpl")
```

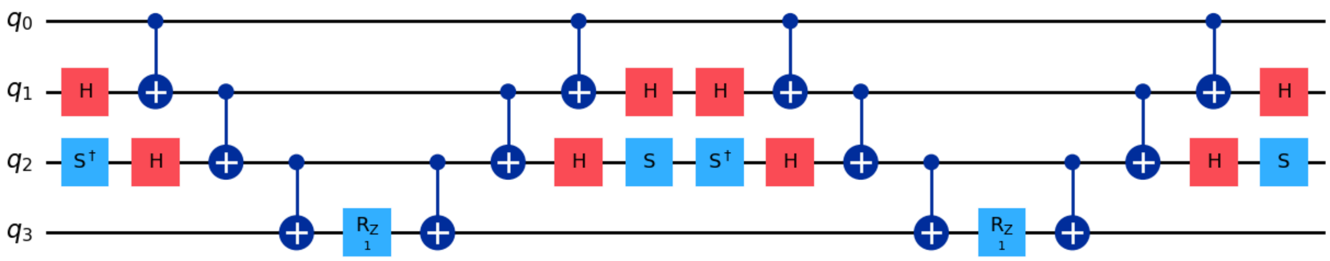
Out[90]:



Circuit as desired. Now confirm if second order is correct.

In [91]:
```
qc=trotter_second_order(ps,t,1,4)
qc.draw("mpl")
```

Out[91]:



Circuit as desired.

(b)

H_2 molecule.

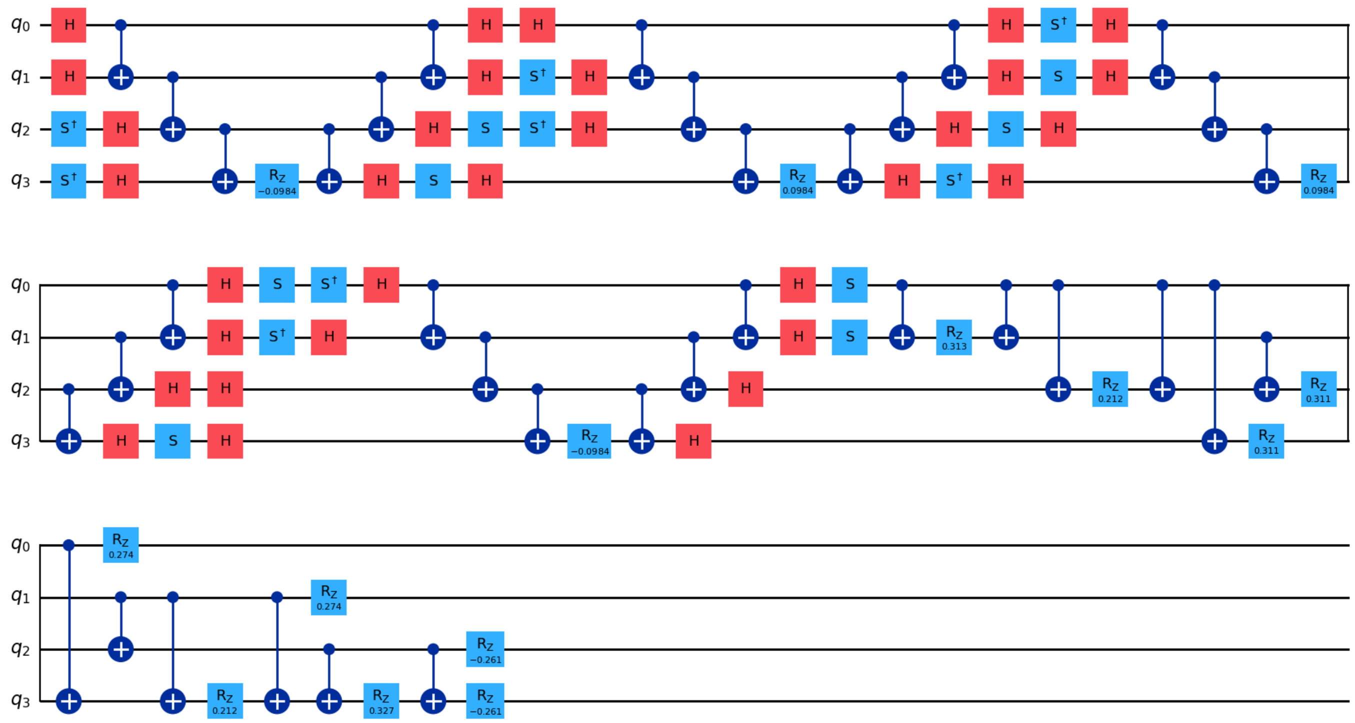
In [92]:
```
#useful for final module
def ps_convert(H_q):
    ops = []
    for term, coeff in H_q._qop.terms.items():
        if term:
            ops.append((coeff.real, PauliString(list(term))))
    return QubitOperator(ops)
H2_q=ps_convert(H_q)
print(H2_q)
```

```
-0.049197645871367546 [X0 X1 Y2 Y3] +
0.049197645871367546 [X0 Y1 Y2 X3] +
0.049197645871367546 [Y0 X1 X2 Y3] +
-0.049197645871367546 [Y0 Y1 X2 X3] +
0.13716572937099492 [Z0] +
0.1566006248823794 [Z0 Z1] +
0.10622904490856078 [Z0 Z2] +
0.15542669077992832 [Z0 Z3] +
0.13716572937099483 [Z1] +
0.15542669077992832 [Z1 Z2] +
0.10622904490856078 [Z1 Z3] +
-0.13036292057109045 [Z2] +
0.1632676867356434 [Z2 Z3] +
-0.13036292057109042 [Z3]
```

In [93]:
```
t = 1
r = 1
num_qubits = 4

qc_1st = trotter_first_order(H2_q, t, r, num_qubits)
qc_2nd = trotter_second_order(H2_q, t, r, num_qubits)
print("First Order:")
qc_1st.draw("mpl")
```

```
First Order:
```

Out[93]:

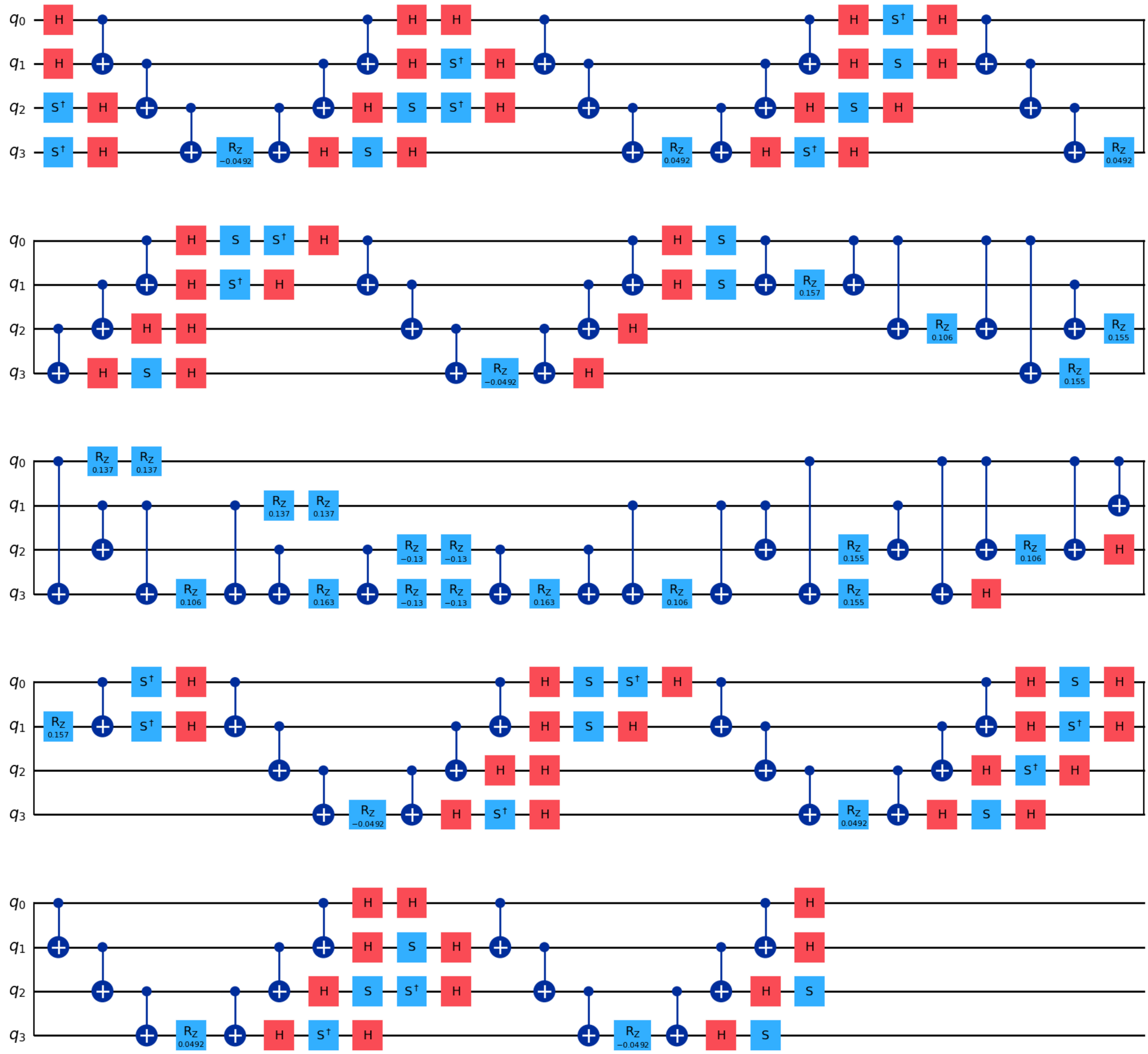


All terms appear as expected, but the one body Z_i terms end up at the end because the conversion rebuilds the operator and can change the term insertion order. Since the Pauli terms do not all commute, different orderings correspond to different first order Trotter approximations, with local error $\mathcal{O}(\Delta t^2)$, which is expected. As long as a single ordering is fixed and used consistently, it is meaningful to compare results across runs (noting that different orderings can have different constants in the error).

In [94]:
```
print("Second Order:")
qc_2nd.draw("mpl")
```

```
Second Order:
```

Out[94]:



Discussion Questions

What is the maximum number of qubits that your computer can handle for state- vector simulations? What aspects of your computer (e.g. CPUs or RAM) determine this? How does this answer change if you change the numerical data type of state-vector elements (e.g. single- or double-precision complex numbers)?

Complex128 is double precision complex and uses 16 bytes per amplitude, and Complex64 is single precision complex and uses 8 bytes per amplitude [7]. My laptop has 24GB (~22.35 GiB) of ram which means with Complex128 I can use 30 qubits $\left(22.35 \cdot 2^{30} \approx 2^{34.48} = 16 \cdot 2^{30.48}\right)$, and with Complex64 I can use 31 qubits $\left(2^{34.48} = 8 \cdot 2^{31.48}\right)$ [8].

Can you give a bound for the depth of the circuit for $\exp(-it\hat{P})$ in terms of the length of $\hat{P}$?

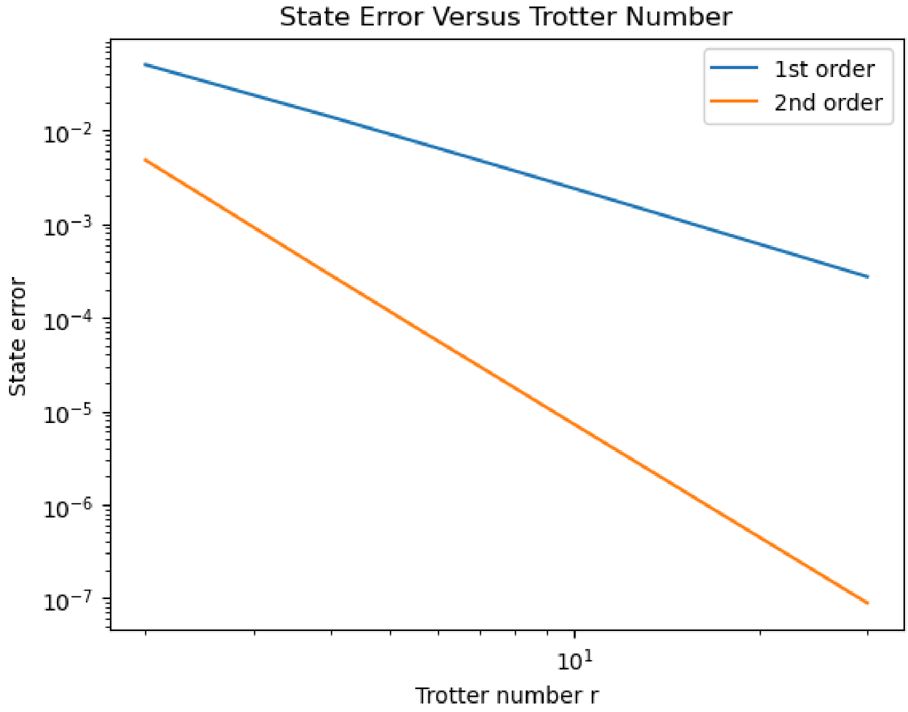
Pauli of length l (l non-identity Paulis) has circuit depth that scales linearly in l. Parity compute uses $l-1$ CNOT layers, then one R_z, then $l-1$ layers to uncompute, worst case Y basis change requires 4 layers $\Rightarrow \text{depth} \leq 2(l-1)+1+4 = 2l+3$.

How does the depth of the circuit for $\exp(-it\hat{P})$ vary with the Trotter number r? What about the numerical errors arising from the Trotter–Suzuki decompositions? Can you quantify this numerically for a simple $\hat{Q}$?

For $\hat{Q} = \sum_{k=0}^{K-1} c_k \hat{P}_k$, first order $\text{depth} \approx r \sum_{k=0}^{K-1} \text{depth}\left[\exp(-i\frac{t}{r} c_k \hat{P}_k)\right] \approx r \sum_{k=0}^{K-1}(2l_k+3)$, and for second order, $\text{depth} \approx 2r \sum_{k=0}^{K-1}(2l_k+3)$.

Error scales as shown below, with example $\hat{Q} = \hat{X}_0 + \hat{Z}_0$. Noting that if A is Hermitian and has eigenvalue equation $Av_i = \lambda_i v_i$ then $A = V\Lambda V^\dagger$ with $\Lambda = \text{diag}(\lambda_i)$, and $V = [v_1 \cdots v_n]$ (for n eigenvectors). This means that $e^{-itA} = e^{-itV\Lambda V^\dagger} = Ve^{-it\Lambda}V^\dagger = V\text{diag}\left(e^{-it\lambda_i}\right)V^\dagger$ (as $V^\dagger V = I$, and Λ is diagonal).

In [95]:
```
t = 1
rs = np.arange(2, 31, 2)
# exact
evals, evecs = np.linalg.eigh(X +Z)
Ue = evecs @ np.diag(np.exp(-1j * t* evals)) @ evecs.conj().T

# trotterisation
Xps = PauliString([(0,"X")])
Zps = PauliString([(0,"Z")])
Q = QubitOperator([(1.0,Xps), (1.0,Zps)])


psi0 =Statevector.from_label("0")
#input state
err1, err2 = [], []
for r in rs:
    qc1 = trotter_first_order(Q, t,r,num_qubits=1)
    qc2 = trotter_second_order(Q, t,r,num_qubits=1)

    psi1 =psi0.evolve(qc1).data
    psi2 =psi0.evolve(qc2).data

    err1.append(1 -np.abs(np.vdot(Ue @ np.array([1,0],complex), psi1))**2)
    #Ue|0>, inner product of this with psi1
    err2.append(1 -np.abs(np.vdot(Ue @ np.array([1,0],complex), psi2))**2)

plt.loglog(rs, err1, label="1st order")
plt.loglog(rs, err2, label="2nd order")
plt.xlabel("Trotter number r")
plt.ylabel(r"State error")
plt.title("State Error Versus Trotter Number")
plt.legend()
plt.show()
```



First order has global operator error $\mathcal{O}(t^2/r) \Rightarrow$ state infidelity scales as $\approx \mathcal{O}(1/r^2)$ (for fixed t), whereas second order has global operator error $\mathcal{O}(t^3/r^2)$ so state infidelity scales as $\approx \mathcal{O}(1/r^4)$. $\Rightarrow$ Expect 1st order to have slope ≈ -2, and 2nd order to have slope ≈ -4.

In [96]:
```
m1, b1 = np.polyfit(np.log(rs), np.log(err1), 1)
m2, b2 = np.polyfit(np.log(rs), np.log(err2), 1)

print("slope (1st order):", m1)
print("slope (2nd order):", m2)
```

```
slope (1st order): -1.9394583122114095
slope (2nd order): -4.0196902494381455
```

Answer as expected.

Module 3 Logbook

This laboratory focused on simulation in quantum computing, both as classical emulation of quantum circuits and as time evolution implemented with qubits. The first part considered classical state-vector simulation. A computational basis bitstring was mapped to an M index complex tensor representing the full 2^M dimensional state. This showed how amplitudes and measurement frequencies are stored and why memory grows exponentially with qubit number.

Single Pauli strings were then applied directly to the state-vector using tensor contractions with the Pauli matrices reproducing simple quantum circuits only using linear algebra. Frequencies obtained from $|V_{i_0 i_1 \cdots i_{M-1}}|^2$ matched the expected computational basis outcomes, which confirmed that the indexing and contraction logic were correct.

The same tasks were repeated using Qiskit. Circuits were initialized to chosen bitstrings and layers of Pauli gates were appended from a PauliString description. Statevector simulation and shot based sampling with the Aer simulator gave the same results as the manual tensor approach.

The next part considered unitary time evolution under a Pauli string $\exp(-it\hat{P})$. Basis changes converted $\hat{X}$ and $\hat{Y}$ operators to $\hat{Z}$, followed by parity accumulation with CNOT ladders, a single $R_z(2t)$ rotation, and uncomputation.

Finally, Trotterisation was used to approximate time evolution generated by a sum of Pauli strings. First and second order Trotter-Suzuki formulas were implemented and applied to the H_2 qubit Hamiltonian. Increasing the Trotter number reduced approximation error but increased circuit depth, which would increase hardware noise on current hardware. A one qubit example with $\hat{Q} = \hat{X} + \hat{Z}$ was used to study the error scaling by comparing Trotterized evolution to the exact solution.

Part IV

Obtaining Electronic Energies

Quantum phase estimation

The electronic Hamiltonian $\hat{H}$ is Hermitian but not unitary, so the unitary operator $\hat{U}(t) = e^{-it\hat{H}}$ is used instead of $\hat{H}$ itself. If $\hat{H}\Psi_j = E_j\Psi_j$, then Ψ_j is also an eigenstate of $\hat{U}(t)$ with eigenvalue e^{-itE_j}, so estimating the phase of $\hat{U}(t)$ can be used to find the energy E_j.

More generally, for a unitary $\hat{U}$ with

$$\hat{U}|\Psi_j\rangle = e^{2\pi i\theta_j}|\Psi_j\rangle, \qquad \theta_j \in [0,1),$$

quantum phase estimation (QPE) uses an n-qubit estimation register to infer the binary fraction $\theta_j \approx \overline{0.b_1 b_2 \cdots b_n} \equiv \frac{\bar{\theta}_j}{2^n}, \quad b_i \in \{0,1\}.$

An example for $\theta_j = 0.5$ is shown below.

In [97]:
```
Image('6_a.jpg')
```

Out[97]:

$$\hat{U}\Psi_j = e^{2\pi i\theta_j}\Psi_j, \qquad \theta_j \in [0,1)$$

$$\theta_j = \underbrace{\overline{0.\,b_1 b_2 \ldots b_n}}_{n \text{ digits}} = \sum_{i=1}^{n} b_i \times 2^{-i} = \frac{1}{2^n}\sum_{i=1}^{n} b_i \times 2^{n-i} \equiv \frac{\bar{\theta}_j}{2^n}, \quad b_i \in \{0,1\},$$

e.g. for $\theta_j = 0.5$, $n = 3$,

$0.5 = \overline{0.100} = 1(2^{-1}) + 0(2^{-2}) + 0(2^{-3}) = \frac{1}{8}(1(2)^2 + 0(2)^1 + 0(2)^0) = \frac{1}{2}$

$\Rightarrow \bar{\theta}_j = 1(2)^2 + 0(2)^1 + 0(2)^0 = 4 = \overline{100}$

for $\theta_j = 0.5$, $|\theta_j\rangle = \sum_{l=0}^{2^n-1} \delta_{\bar{\theta}_j l}|l\rangle = \underbrace{\delta_{4,0}|0\rangle + \delta_{4,1}|1\rangle + \delta_{4,2}|2\rangle + \delta_{4,3}|3\rangle}_{0} + \underbrace{\delta_{4,4}}_{1}|4\rangle + \underbrace{\ldots + \delta_{4,7}|7\rangle}_{0}$

$\Rightarrow |\theta_j\rangle = |4\rangle = |100\rangle$

The QPE circuit prepares a uniform superposition on the estimation register. Controlled powers $\hat{U}^{2^m}$ are applied to the state register (initialised as $|\Psi_j\rangle$), controlled on each estimation qubit ($m = 0, 1, \cdots, n-1$). A QFT^{-1} is then applied, noting that $\mathrm{QFT}|\theta_j\rangle = \frac{1}{\sqrt{2^n}}\sum_{k=0}^{2^n-1} e^{2\pi i\theta_j k}|k\rangle$. The estimation register then becomes the state $|\theta_j\rangle$.

If the state register is initialised in a guess state

$$|\Psi_{\text{guess}}\rangle = \sum_j c_j |\Psi_j\rangle,$$

then an estimate of θ_j accurate to n binary places is obtained with probability $|c_j|^2$, so good overlap with the desired eigenstate is required [1].

In [98]:
```
Image('4_1a.png')
```

Out[98]:

Task IV.1: Quantum phase estimation.

You will need to consult the relevant API documentations for your quantum-computing library of choice such as Qiskit to complete this task.

Using a high-level quantum-computing library, write code to carry out the QPE procedure described above. In particular, your code should:

- define M qubits for the state register and n qubits for the estimation register,

In [99]:
```
Image('4_1b.png')
```

Out[99]:

- take in a circuit representing the unitary operator $\hat{U}$ acting on the state register and construct the necessary controlled-unitary circuits,
- make use of the QFT^{-1} circuit provided by your quantum-computing library of choice to avoid having to program this from scratch,
- output the circuit depth, and
- output the most probable phase θ_j as a decimal fraction and its frequency.

In [100…
```
def build_qpe(U,n):
    M= U.num_qubits
    est= QuantumRegister(n, "est")
    state= QuantumRegister(M, "state")
    c= ClassicalRegister(n, "c")
    qc= QuantumCircuit(state, est, c)

    qc.h(est)
    # hadamards to estimation register
    Ug = U.to_gate(label="U")
    for m in range(n):
        Upow = Ug.power(2**m).control(1)
        #1 control qubit
        qc.append(Upow, [est[m]] + list(state))
        #[control, state[0],..., state[M-1]]

    qc.append(QFTGate(n).inverse(), est)
    qc.measure(est, c)
    return qc

backend=AerSimulator()
def run_qpe(U, n, prep_state, shots,num_theta=3,draw_circuit=False,print_=False):
    M= U.num_qubits
    qc= build_qpe(U, n)

    prep = QuantumCircuit(n + M, n)
    #n+M qubits, n classical bits
    prep.compose(prep_state, qubits=list(range(0,M)), inplace=True)
    # state register first
    qc = prep.compose(qc)
    if draw_circuit:
        print(qc.draw(output="text"))

    transp = transpile(qc,backend)

    t0 = time.perf_counter()
```

```
    counts = backend.run(transp, shots=shots).result().get_counts()

    time_taken = time.perf_counter() - t0

# commented out block was used initially finding the most frequent bit string, but IV.3(a)(i)
# motivated finding the three most frequent bit strings

#    count, bitstr = max((c, b) for b, c in counts.items())
#    freq = count / shots
#    #bitstring with biggest count
#    theta = int(bitstr.replace(" ", ""), 2) / (2**n)


    top = sorted(counts.items(), key=lambda x: x[1], reverse=True)[:num_theta] #sort by freq

    thetas = []
    freqs = []
    for bitstr, count in top:
        thetas.append(int(bitstr.replace(" ", ""), 2) / (2**n))#base 2
        freqs.append(count / shots)



    if print_:
        print("Transpiled circuit depth:", transp.depth())
        print("Time taken (s):", time_taken)
        print("Most probable theta:", thetas[0])
        print("Frequency:", freqs[0])

    return qc, counts, transp.depth(), time_taken,thetas
```

Does this produce the same result as the handwritten result for $\theta_j = 0.5$?

In [101…

```
theta_true = 1/2
U = QuantumCircuit(1)
U.p(2*np.pi*theta_true, 0)

prep = QuantumCircuit(1)
#P|1>=e^(i phi) |1>
prep.x(0)

run_qpe(U, n=3, prep_state=prep, shots=10000,draw_circuit=True,print_=True)
```

```
     ┌───┐┌────────────────┐┌────────────────┐┌────────────────┐                  »
q_0: ┤ X ├┤ circuit-1643^1 ├┤ circuit-1643^2 ├┤ circuit-1643^4 ├──────────────────»
     ├───┤└───────┬────────┘└───────┬────────┘└───────┬────────┘┌─────────┐┌─┐    »
q_1: ┤ H ├────────■─────────────────┼───────────────────┼─────────┤0        ├┤M├────»
     ├───┤                          │                   │         │         │└╥┘    »
q_2: ┤ H ├──────────────────────────■───────────────────┼─────────┤1 qft_dg ├─╫─────»
     ├───┤                                              │         │         │ ║     »
q_3: ┤ H ├──────────────────────────────────────────────■─────────┤2        ├─╫─────»
     └───┘                                                        └─────────┘ ║     »
c: 3/═════════════════════════════════════════════════════════════════════════╩═════»
                                                                              0     »
«            
«q_0: ──────
«            
«q_1: ──────
«     ┌─┐   
«q_2: ┤M├───
«     └╥┘┌─┐
«q_3: ─╫─┤M├
«      ║ └╥┘
«c: 3/═╩══╩═
«      1  2 
Transpiled circuit depth: 7
Time taken (s): 0.00277758299995412
Most probable theta: 0.5
Frequency: 1.0
```

Out[101…

```
(<qiskit.circuit.quantumcircuit.QuantumCircuit at 0x321eb0680>,
 {'100': 10000},
 7,
 0.00277758299995412,
 [0.5])
```

The agreement is good, and the circuit looks as expected

Now consider $\theta_j = \frac{3}{8}$.

In [102…

```python
theta_true = 3/8
U = QuantumCircuit(1)
U.p(2*np.pi*theta_true, 0)

prep = QuantumCircuit(1)
#P|1>=e^(i phi) |1>
prep.x(0)

run_qpe(U, n=3, prep_state=prep, shots=10000,draw_circuit=True,print_=True)
```

```
     ┌───┐┌────────────────┐┌────────────────┐┌────────────────┐               »
q_0: ┤ X ├┤ circuit-1660^1 ├┤ circuit-1660^2 ├┤ circuit-1660^4 ├───────────────»
     ├───┤└───────┬────────┘└───────┬────────┘└───────┬────────┘┌─────────┐┌─┐»
q_1: ┤ H ├────────■─────────────────┼───────────────────┼─────────┤0        ├┤M├»
     ├───┤                          │                   │         │         │└╥┘»
q_2: ┤ H ├──────────────────────────■───────────────────┼─────────┤1 qft_dg ├─╫─»
     ├───┤                                              │         │         │ ║ »
q_3: ┤ H ├──────────────────────────────────────────────■─────────┤2        ├─╫─»
     └───┘                                                        └─────────┘ ║ »
c: 3/═════════════════════════════════════════════════════════════════════════╩═»
                                                                              0 »
«
«q_0: ──────
«
«q_1: ──────
«     ┌─┐
«q_2: ┤M├───
«     └╥┘┌─┐
«q_3: ─╫─┤M├
«      ║ └╥┘
«c: 3/═╩══╩═
«      1  2
Transpiled circuit depth: 16
Time taken (s): 0.003332915999976649
Most probable theta: 0.375
Frequency: 1.0
```

Out[102…

```
(<qiskit.circuit.quantumcircuit.QuantumCircuit at 0x321eb28e0>,
 {'011': 10000},
 16,
 0.003332915999976649,
 [0.375])
```

Good agreement.

Hamiltonian eigenvalues from quantum phase estimation

When QPE is applied to $\hat{U}(t) = e^{-it\hat{H}}$, an eigenstate Ψ_j gives an eigenvalue phase

$$e^{-itE_j} = e^{2\pi i\theta_j}, \qquad \theta_j \in [0,1).$$

Phases are defined modulo 2π, so the energy is only determined up to an integer branch,

$$E_j = -\frac{2\pi}{t}(\theta_j + m), \qquad m \in \mathbb{Z}^{0+}, t \in \mathbb{R}^+,$$

with m restricted to non-negative integers and $t > 0$ to ensure $E_j \leq 0$. To obtain a unique E_j, choose energy bounds $E_{j,\min} \leq E_j < E_{j,\max}$ and set t so that each branch interval

$$-\frac{2\pi(m+1)}{t} \leq E_j < -\frac{2\pi m}{t}$$

has width equal to the allowed energy range,

$$\frac{2\pi}{t} = E_{j,\max} - E_{j,\min}.$$

Then, provided the true energy lies within the chosen bounds, each measured θ_j corresponds to a unique integer m, and hence to a unique E_j in $[E_{j,\min}, E_{j,\max})$ [1].

In [103…

```python
Image('4_2a.png')
```

Out[103…

Task IV.2: QPE for the ground energy of H_2.

(a) *State preparation.* Use the code written in Task II.5 to construct a circuit that brings the state register from the initial vacuum state $|\text{vac}\rangle$ to the ground UHF state $|\Psi^{\text{UHF}}\rangle$ of the H_2 system computed in Task II.3(b). This serves as the guess state in Equation (6.9) and we expect that the ground UHF state should have a large overlap with the true electronic ground state of the system, so that the obtained energy from QPE will in fact be the ground energy E_0.

(b) *Main QPE procedure.*

(i) Use the quantum circuit for the Trotterised time-evolved qubit Hamiltonian of H_2 obtained in Task III.4(b) as the unitary-operator input for the QPE code written in Task IV.1.

(ii) Use the UHF energy as a guide, decide on suitable bounds for E_0. Then, use Equation (6.12) to compute the corresponding t.

(iii) Simulate the QPE circuit with a moderate estimation register size ($n = 5 \sim 8$) and the above value of t to obtain θ_0 together with its frequency.

(c) *Energy computation.*

(i) From the energy bounds selected above, use Equation (6.11) to find an integer value of m that gives rise to the unique E_0 that lies within the bounds.

(ii) Compare the obtained E_0 with the UHF energy computed in Task II.3(b).

(iii) Using your electronic-structure package of choice, compute the ground full configuration interaction (FCI) energy for the same H_2 system and compare it with the obtained E_0.

(a)

The occupied orbitals are taken from II.3(b).

In [130…

```
atom = """
H 0 0 0
H 0 0 1
"""
C_a, C_b, labels, (occ_a, occ_b),E_uhf = uhf_mos(atom, "sto-3g")
print("Occupied alpha sites =", occ_a)
print("Occupied beta sites =", occ_b)
```

```
Occupied alpha sites = [1. 0.]
Occupied beta sites = [1. 0.]
```

In [131…

```
k_uhf = (1, 1, 0, 0)
ps = occupation_to_paulistring(k_uhf)
prep_state = QuantumCircuit(4)
append_pauli(prep_state, ps)

print(prep_state)
```

```
     ┌───┐
q_0: ┤ X ├
     ├───┤
q_1: ┤ X ├
     └───┘
q_2: ─────

q_3: ─────
```

Produces $|0011\rangle$, as desired.

(b)

(ii)

Picking suitable energy bounds is highly non-trivial. One practical choice is to use the Hartree-Fock energy as an upper bound. Since Hartree-Fock minimizes the energy within a restricted variational space, the variational principle ensures that E_{HF} is an upper bound to the exact ground-state energy of the Hamiltonian [9]. In the same finite orbital basis, full configuration interaction (FCI) gives the exact ground-state energy of the Hamiltonian in that basis, so $E_0 = E_{\mathrm{FCI}} \leq E_{\mathrm{HF}}$ [10]. To allow for small numerical error and make sure that the true energy lies within the interval, $E_{\min}$ is taken as $1.001E_{\mathrm{FCI}}$.

Note that all energies are in Hartree, E_{h}, with numerical value $4.3597447222060 \times 10^{-18}$ J [11].

In [134…
```
print("UHF total energy =", E_uhf, "Hartree")
Emax=E_uhf
```

```
UHF total energy = -1.0661086493179366 Hartree
```

The full configuration interaction energy is calculated below [12]. Note that the unrestricted Hartree-Fock energy will be used for the rest of the notebook.

In [135…
```
mol = gto.M(atom=atom, basis=basis,unit="Angstrom",charge=0,spin=0,verbose=0)
mf = scf.UHF(mol).run() #unrestricted works

cisolver = fci.FCI(mol, mf.mo_coeff)
EFCI=cisolver.kernel()[0]
Emin=1.001*EFCI

print("FCI energy =", EFCI, "Hartree")
```

```
FCI energy = -1.1011503302326187 Hartree
```

t can now be found with $E_{\max} = E_{\mathrm{HF}}$ and $E_{\min} = 1.001 * E_{\mathrm{FCI}}$.

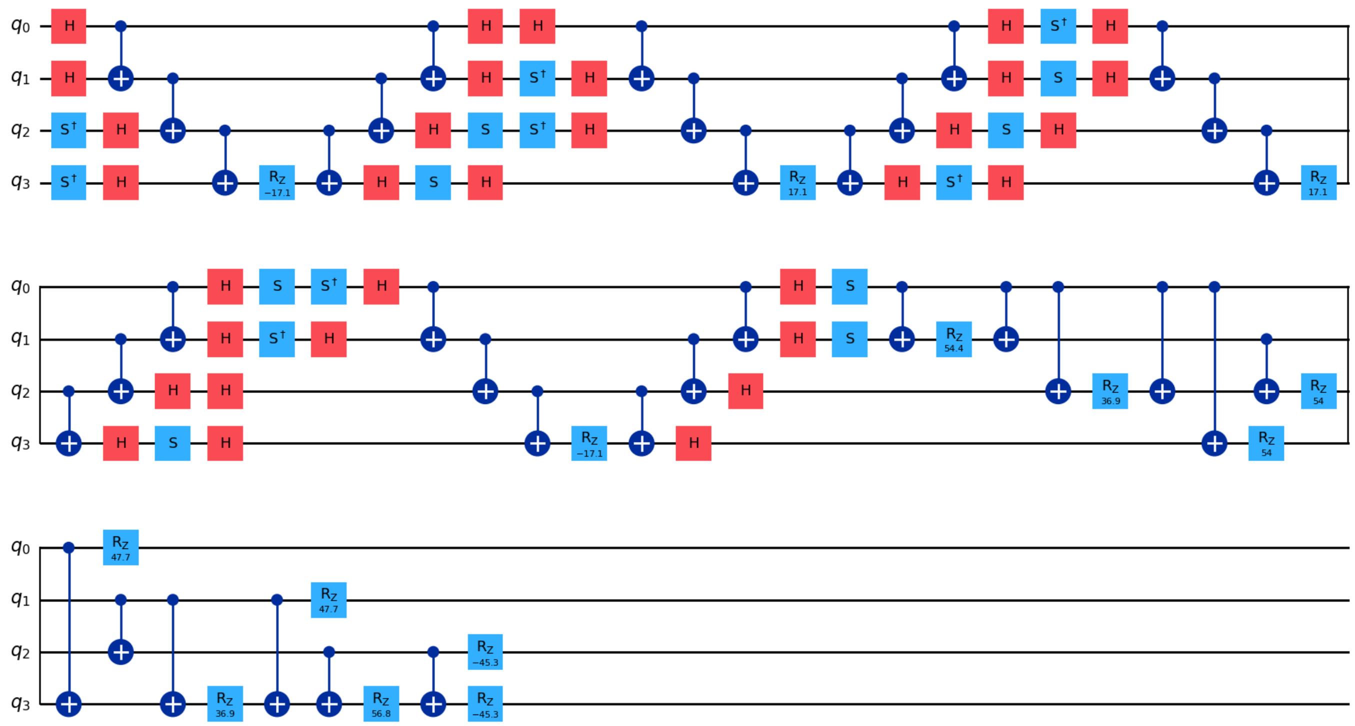
In [136…
```
t=2*np.pi/(Emax-Emin)
print("t =", t)
```

```
t = 173.84319630642253
```

(i)

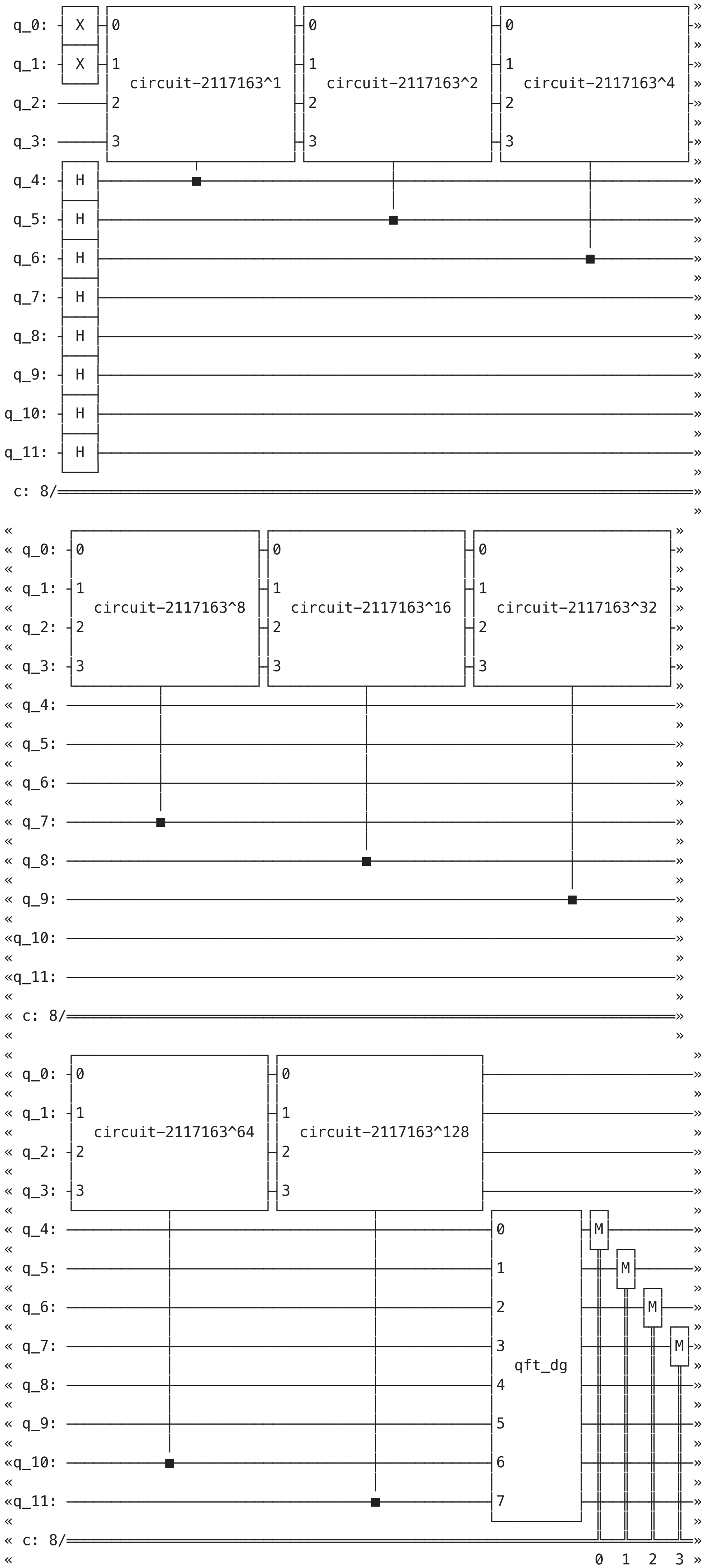
In [137…
```
r=1
num_qubits = 4

U_1 = trotter_first_order(H2_q, t, r, num_qubits)
U_1.draw("mpl")
```

Out[137…



(iii)

In [138…
```
qc_qpe, counts, depth, time_taken,thetas = run_qpe(U_1, n=8, prep_state=prep_state, shots=10000, draw_circuit=
```


q_0:
q_1:
q_2:
q_3:
q_4:
q_5:
q_6:
q_7:
q_8:
q_9:
q_10:
q_11:
c: 8/
X
H
circuit-2117163^1
circuit-2117163^2
circuit-2117163^4
circuit-2117163^8
circuit-2117163^16
circuit-2117163^32
circuit-2117163^64
circuit-2117163^128
qft_dg
M
0 1 2 3

```
«
« q_0: ────────────
«
« q_1: ────────────
«
« q_2: ────────────
«
« q_3: ────────────
«
« q_4: ────────────
«
« q_5: ────────────
«
« q_6: ────────────
«
« q_7: ────────────
«      ┌─┐
« q_8: ┤M├─────────
«      └╥┘┌─┐
« q_9: ─╫─┤M├──────
«       ║ └╥┘┌─┐
«q_10: ─╫──╫─┤M├───
«       ║  ║ └╥┘┌─┐
«q_11: ─╫──╫──╫─┤M├
«       ║  ║  ║ └╥┘
« c: 8/═╩══╩══╩══╩═
«       4  5  6  7
Transpiled circuit depth: 3647
Time taken (s): 0.051289541999949506
Most probable theta: 0.9296875
Frequency: 0.9609
```

In [139...
```python
print("Most common bitstring:", max(counts, key=counts.get))
```

```
Most common bitstring: 11101110
```

(c)

(i)

Code that finds E_{QPE} automatically is useful for the next tasks.

In [140...
```python
def find_E(theta, Emin,Emax):
    t=2*np.pi / (Emax - Emin)
    mlow= int(np.ceil(((-Emax) * t / (2*np.pi)) - theta))
    mhigh = int(np.floor(((-Emin) * t / (2*np.pi)) - theta))
    if mlow != mhigh:
        raise ValueError("Change bounds")
    m=mhigh
    E = -(2*np.pi / t) * (theta + m)
    return E, m
EQPE,m=find_E(thetas[0],Emin,Emax)
print("m =",m)
print("EQPE =",EQPE, "Hartree")
```

```
m = 29
EQPE = -1.0817436445255293 Hartree
```

(ii)

In [141...
```python
print("Hartree-Fock energy is",E_uhf, "Hartree")
```

```
Hartree-Fock energy is -1.0661086493179366 Hartree
```

The E_{QPE} value found is lower than the Hartree-Fock energy, as expected for a more accurate approximation to the ground-state energy.

(iii)

In [142...
```python
print("FCI energy =", EFCI, "Hartree")
```

```
FCI energy = -1.1011503302326187 Hartree
```

The FCI energy is still lower, so using second order Trotterisation is one possible way to improve the estimate.

In [143...
```python
U_2 = trotter_second_order(H2_q, t, r, num_qubits)
qc_qpe, counts, depth, time_taken,thetas = run_qpe(U_2, n=8, prep_state=prep_state, shots=10000, draw_circuit=
```

```
      ┌───┐┌──────────────────────┐┌──────────────────────┐┌──────────────────────┐»
 q_0: ┤ X ├┤0                     ├┤0                     ├┤0                     ├»
      ├───┤│                      ││                      ││                      │»
 q_1: ┤ X ├┤1                     ├┤1                     ├┤1                     ├»
      └───┘│  circuit-2119638^1   ││  circuit-2119638^2   ││  circuit-2119638^4   │»
 q_2: ─────┤2                     ├┤2                     ├┤2                     ├»
           │                      ││                      ││                      │»
 q_3: ─────┤3                     ├┤3                     ├┤3                     ├»
      ┌───┐└──────────┬───────────┘└──────────┬───────────┘└──────────┬───────────┘»
 q_4: ┤ H ├───────────■───────────────────────┼───────────────────────┼────────────»
      ├───┤                                   │                       │            »
 q_5: ┤ H ├───────────────────────────────────■───────────────────────┼────────────»
      ├───┤                                                           │            »
 q_6: ┤ H ├───────────────────────────────────────────────────────────■────────────»
      ├───┤                                                                        »
 q_7: ┤ H ├────────────────────────────────────────────────────────────────────────»
      ├───┤                                                                        »
 q_8: ┤ H ├────────────────────────────────────────────────────────────────────────»
      ├───┤                                                                        »
 q_9: ┤ H ├────────────────────────────────────────────────────────────────────────»
      ├───┤                                                                        »
q_10: ┤ H ├────────────────────────────────────────────────────────────────────────»
      ├───┤                                                                        »
q_11: ┤ H ├────────────────────────────────────────────────────────────────────────»
      └───┘                                                                        »
 c: 8/═════════════════════════════════════════════════════════════════════════════»
                                                                                   »
«      ┌─────────────────────┐┌──────────────────────┐┌──────────────────────┐»
« q_0: ┤0                    ├┤0                     ├┤0                     ├»
«      │                     ││                      ││                      │»
« q_1: ┤1                    ├┤1                     ├┤1                     ├»
«      │  circuit-2119638^8  ││  circuit-2119638^16  ││  circuit-2119638^32  │»
« q_2: ┤2                    ├┤2                     ├┤2                     ├»
«      │                     ││                      ││                      │»
« q_3: ┤3                    ├┤3                     ├┤3                     ├»
«      └──────────┬──────────┘└──────────┬───────────┘└──────────┬───────────┘»
« q_4: ───────────┼──────────────────────┼───────────────────────┼────────────»
«                 │                      │                       │            »
« q_5: ───────────┼──────────────────────┼───────────────────────┼────────────»
«                 │                      │                       │            »
« q_6: ───────────┼──────────────────────┼───────────────────────┼────────────»
«                 │                      │                       │            »
« q_7: ───────────■──────────────────────┼───────────────────────┼────────────»
«                                        │                       │            »
« q_8: ──────────────────────────────────■───────────────────────┼────────────»
«                                                                │            »
« q_9: ──────────────────────────────────────────────────────────■────────────»
«                                                                             »
«q_10: ───────────────────────────────────────────────────────────────────────»
«                                                                             »
«q_11: ───────────────────────────────────────────────────────────────────────»
«                                                                             »
« c: 8/═══════════════════════════════════════════════════════════════════════»
«                                                                             »
«      ┌──────────────────────┐┌───────────────────────┐                         »
« q_0: ┤0                     ├┤0                      ├─────────────────────────»
«      │                      ││                       │                         »
« q_1: ┤1                     ├┤1                      ├─────────────────────────»
«      │  circuit-2119638^64  ││  circuit-2119638^128  │                         »
« q_2: ┤2                     ├┤2                      ├─────────────────────────»
«      │                      ││                       │                         »
« q_3: ┤3                     ├┤3                      ├─────────────────────────»
«      └──────────┬───────────┘└──────────┬────────────┘┌─────────┐┌─┐           »
« q_4: ───────────┼───────────────────────┼─────────────┤0        ├┤M├───────────»
«                 │                       │             │         │└╥┘┌─┐        »
« q_5: ───────────┼───────────────────────┼─────────────┤1        ├─╫─┤M├────────»
«                 │                       │             │         │ ║ └╥┘┌─┐     »
« q_6: ───────────┼───────────────────────┼─────────────┤2        ├─╫──╫─┤M├─────»
«                 │                       │             │         │ ║  ║ └╥┘┌─┐  »
« q_7: ───────────┼───────────────────────┼─────────────┤3        ├─╫──╫──╫─┤M├──»
«                 │                       │             │  qft_dg │ ║  ║  ║ └╥┘  »
« q_8: ───────────┼───────────────────────┼─────────────┤4        ├─╫──╫──╫──╫───»
«                 │                       │             │         │ ║  ║  ║  ║   »
« q_9: ───────────┼───────────────────────┼─────────────┤5        ├─╫──╫──╫──╫───»
«                 │                       │             │         │ ║  ║  ║  ║   »
«q_10: ───────────■───────────────────────┼─────────────┤6        ├─╫──╫──╫──╫───»
«                                         │             │         │ ║  ║  ║  ║   »
«q_11: ───────────────────────────────────■─────────────┤7        ├─╫──╫──╫──╫───»
«                                                       └─────────┘ ║  ║  ║  ║   »
« c: 8/═════════════════════════════════════════════════════════════╩══╩══╩══╩═══»
«                                                                   0  1  2  3   »
```

```
«
« q_0: ────────────
«
« q_1: ────────────
«
« q_2: ────────────
«
« q_3: ────────────
«
« q_4: ────────────
«
« q_5: ────────────
«
« q_6: ────────────
«
« q_7: ────────────
«      ┌─┐
« q_8: ┤M├─────────
«      └╥┘┌─┐
« q_9: ─╫─┤M├──────
«       ║ └╥┘┌─┐
«q_10: ─╫──╫─┤M├───
«       ║  ║ └╥┘┌─┐
«q_11: ─╫──╫──╫─┤M├
«       ║  ║  ║ └╥┘
« c: 8/═╩══╩══╩══╩═
«       4  5  6  7
Transpiled circuit depth: 3647
Time taken (s): 0.04731866700012688
Most probable theta: 0.328125
Frequency: 0.9941
```

In [144…

```python
EQPE,m=find_E(thetas[0],Emin,Emax)
print("m =",m)
print("EQPE =",EQPE, "Hartree")
```

```
m = 30
EQPE = -1.0961443038496748 Hartree
```

An improvement is observed. This is somewhat idealized since the calculation uses a noise-free simulator. On real quantum hardware, the increased circuit depth would generally introduce more noise. The printed depths of the two QPE circuits are identical here because the time evolution unitaries are treated as single gates in this construction. The depth difference between the first order and second order time evolution circuits used to construct U is shown below.

In [145…

```python
print("First order depth:",U_1.depth())
print("Second order depth:",U_2.depth())
```

```
First order depth: 57
Second order depth: 114
```

(d)

(i)

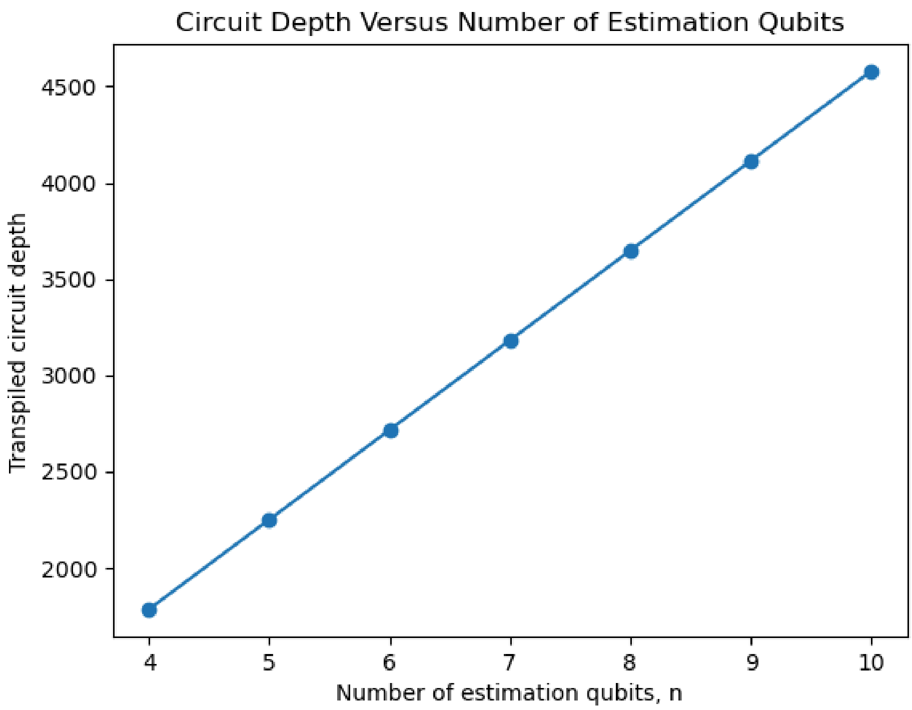
Included above.

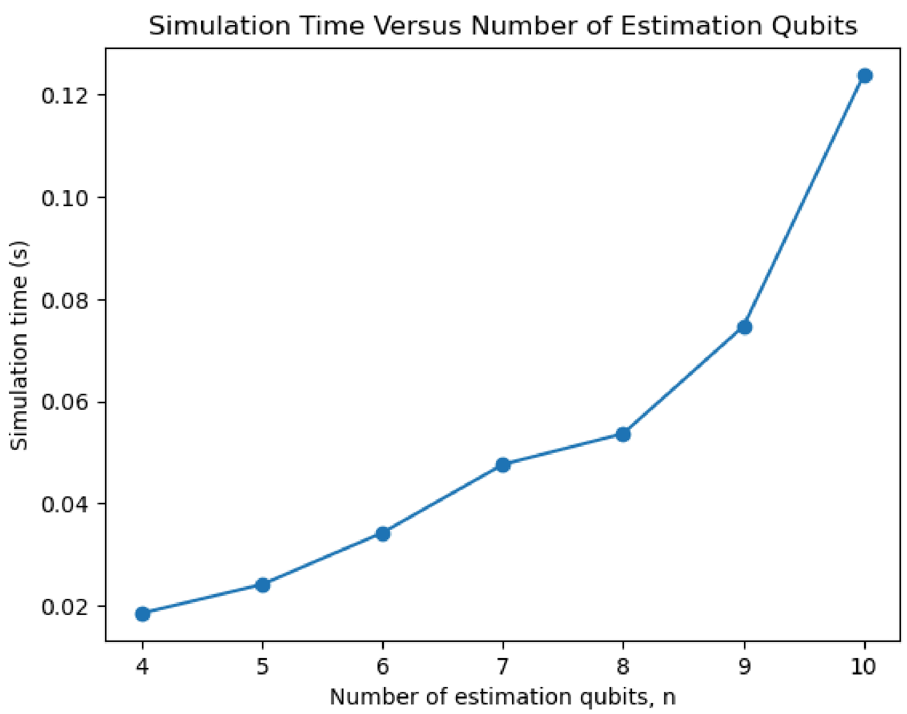
(ii)

In [118…

```python
depths = []
times = []

n_list = list(range(4, 11))

for n in n_list:
    mult_run_time=[]
    for r in range(100):#take the average time over many runs to reduce timing noise
        qc_qpe, counts, depth, time_taken,thetas = run_qpe(U_1, n=n, prep_state=prep_state, shots=10000, draw_
        mult_run_time.append(time_taken)
    depths.append(depth)
    times.append(np.mean(mult_run_time))

plt.figure()
plt.plot(n_list, depths, marker="o")
plt.xlabel("Number of estimation qubits, n")
plt.ylabel("Transpiled circuit depth")
plt.title("Circuit Depth Versus Number of Estimation Qubits")
plt.show()

plt.figure()
```

```python
plt.plot(n_list, times, marker="o")
plt.xlabel("Number of estimation qubits, n")
plt.ylabel("Simulation time (s)")
plt.title("Simulation Time Versus Number of Estimation Qubits")
plt.show()
```





Circuit depth increases approximately linearly with n. Simulation time also increases with n, but more rapidly. This is broadly because a statevector simulation of N qubits stores 2^N amplitudes, so the memory and computational cost grow exponentially with the total qubit number.

In [119…
```python
Image('4_3.png')
```

Out [119…

Task IV.3: Potential energy surfaces of H_2 from QPE.

(a) *Ground-state dissociation curve.*

(i) Repeat Task IV.2 for different bond lengths R_{HH} of H_2 to plot the ground energy E_0 obtained from QPE as a function of R_{HH} between 0.5 Å and 5.0 Å. This is known as the ground-state *dissociation curve* for H_2.

(ii) Also calculate and plot the FCI ground-state dissociation curve for H_2. How does this compare to the QPE equivalent?

(b) *Excited-state dissociation curves.*

(i) By varying the guess state, it is possible to harness QPE to find excited-state energies. Repeat Task IV.2 for several different UHF occupation-number vectors and plot the corresponding QPE dissociation curves. How do these compare with the ground-state one?

(ii) Also calculate and plot FCI excited-state dissociation curves for H_2. How do these compare to the QPE equivalents?

(a)

First compare the H_2 Hamiltonian from the written functions with the PySCF values for extreme R_{HH} values.

In [120…

```
atom = """
H 0 0 0
H 0 0 0.5
"""
print("0.5 Angstrom")
H = second_quantized_hamiltonian(atom=atom, basis="sto-3g")
H_q = jw_transform(H)
print("Written\n")
print(H_q)
mol = MolecularData(atom, basis, multiplicity=1)
mol = run_pyscf(mol)

H_ref = jordan_wigner(mol.get_molecular_hamiltonian())
print("\nPySCF\n")
print(H_ref)
```

```
0.5 Angstrom
Written

(0.3798313517809502+0j) [] +
(-0.04221755692243371+0j) [X0 X1 Y2 Y3] +
(0.04221755692243371+0j) [X0 Y1 Y2 X3] +
(0.04221755692243371+0j) [Y0 X1 X2 Y3] +
(-0.04221755692243371+0j) [Y0 Y1 X2 X3] +
(0.21393531024521223+0j) [Z0] +
(0.17992650976405983+0j) [Z0 Z1] +
(0.13459240346368884+0j) [Z0 Z2] +
(0.1768099603861225+0j) [Z0 Z3] +
(0.21393531024521223+0j) [Z1] +
(0.1768099603861225+0j) [Z1 Z2] +
(0.13459240346368884+0j) [Z1 Z3] +
(-0.3691443152437638+0j) [Z2] +
(0.1862098425924703+0j) [Z2 Z3] +
(-0.36914431524376384+0j) [Z3]

PySCF

0.3798313517809503 [] +
-0.042217556922433716 [X0 X1 Y2 Y3] +
0.042217556922433716 [X0 Y1 Y2 X3] +
0.042217556922433716 [Y0 X1 X2 Y3] +
-0.042217556922433716 [Y0 Y1 X2 X3] +
0.21393531024521256 [Z0] +
0.1799265097640598 [Z0 Z1] +
0.13459240346368873 [Z0 Z2] +
0.17680996038612246 [Z0 Z3] +
0.21393531024521256 [Z1] +
0.17680996038612246 [Z1 Z2] +
0.13459240346368873 [Z1 Z3] +
-0.3691443152437641 [Z2] +
0.18620984259247086 [Z2 Z3] +
-0.3691443152437641 [Z3]
```

In [121…

```python
atom = """
H 0 0 0
H 0 0 5
"""
print("5 Angstrom")
H = second_quantized_hamiltonian(atom=atom, basis="sto-3g")
H_q = jw_transform(H)
print("Written\n")
print(H_q)
mol = MolecularData(atom, basis, multiplicity=1)
mol = run_pyscf(mol)

H_ref = jordan_wigner(mol.get_molecular_hamiltonian())
print("\nPySCF\n")
print(H_ref)
```

```
5 Angstrom
Written

(-0.5458607027942335+0j) [] +
(-0.08359631370988604+0j) [X0 X1 Y2 Y3] +
(0.08359631370988604+0j) [X0 Y1 Y2 X3] +
(0.08359631370988604+0j) [Y0 X1 X2 Y3] +
(-0.08359631370988604+0j) [Y0 Y1 X2 X3] +
(0.03970104575308903+0j) [Z0] +
(0.11004294256569613+0j) [Z0 Z1] +
(0.026458859479781632+0j) [Z0 Z2] +
(0.11005517318966766+0j) [Z0 Z3] +
(0.039701045753089026+0j) [Z1] +
(0.11005517318966766+0j) [Z1 Z2] +
(0.02645885947978162+0j) [Z1 Z3] +
(0.03957781813360586+0j) [Z2] +
(0.11006740930024875+0j) [Z2 Z3] +
(0.039577818133605724+0j) [Z3]

PySCF

-0.5458607027942334 [] +
-0.08359631370988599 [X0 X1 Y2 Y3] +
0.08359631370988599 [X0 Y1 Y2 X3] +
0.08359631370988599 [Y0 X1 X2 Y3] +
-0.08359631370988599 [Y0 Y1 X2 X3] +
0.039701045753089165 [Z0] +
0.11004294256569604 [Z0 Z1] +
0.02645885947978159 [Z0 Z2] +
0.1100551731896676 [Z0 Z3] +
0.03970104575308918 [Z1] +
0.1100551731896676 [Z1 Z2] +
0.02645885947978159 [Z1 Z3] +
0.03957781813360589 [Z2] +
0.11006740930024869 [Z2 Z3] +
0.03957781813360592 [Z3]
```

Good agreement.

(i)

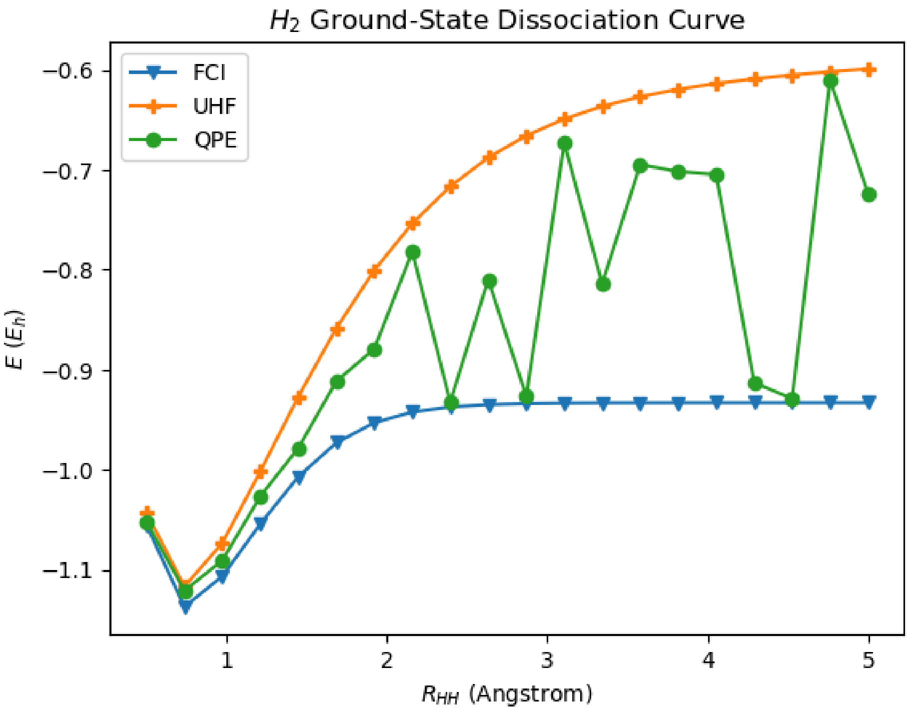
In [122…

```python
def Ecalc(R, prep, n, basis="sto-3g", shots=200000, r=1, trotter_order=1,num_theta=3):
    #geometry
    atom = f"""
    H 0 0 0
    H 0 0 {R}
    """

    #E UHF
    C_a, C_b, labels, (occ_a, occ_b), E_uhf = uhf_mos(atom, basis)

    #prepare state, generalised for part b
    ps = occupation_to_paulistring(prep)
    prep_state = QuantumCircuit(4)
    append_pauli(prep_state, ps)

    #FCI
    mol = gto.M(atom=atom, basis=basis,unit="Angstrom",charge=0,spin=0,verbose=0)
    mf = scf.UHF(mol).run()

    cisolver = fci.FCI(mol, mf.mo_coeff)
    EFCI=cisolver.kernel()[0]
    # bounds
    Emin = 1.001*EFCI
    Emax = E_uhf
    t = 2*np.pi / (Emax - Emin)

    H = second_quantized_hamiltonian(atom=atom, basis=basis)
    H_q = jw_transform(H)
    H2_q=ps_convert(H_q)

    num_qubits = 4
    if trotter_order == 1:
        U = trotter_first_order(H2_q, t, r, num_qubits)
    else:
        U = trotter_second_order(H2_q, t, r, num_qubits)

    #QPE
    qc_qpe, counts, depth, time_taken,thetas = run_qpe(U, n=n, prep_state=prep_state, shots=shots,num_theta=nur
```

```
    # Energies
    energies = [find_E(theta, Emin, Emax)[0] for theta in thetas]
    return energies, EFCI, E_uhf
```

In [124…

```
R_list = np.linspace(0.5, 5.0, 20)
k_uhf = (1, 1, 0, 0)
EQPEs = []
FCIs = []
UHFs=[]
for R in R_list:
    energies,FCI,UHF=Ecalc(R,k_uhf, n=8, basis="sto-3g", shots=10000, r=20, trotter_order=2)
    EQPEs.append(energies[0])
    FCIs.append(FCI)
    UHFs.append(UHF)


plt.figure()
plt.plot(R_list, FCIs, marker="v", label="FCI")
plt.plot(R_list, UHFs, marker="P", label="UHF")
plt.plot(R_list, EQPEs, marker="o", label="QPE")
plt.xlabel(r"$R_{HH}$ (Angstrom)")
plt.ylabel(r"$E$ ($E_h$)")
plt.title("$H_2$ Ground-State Dissociation Curve")
plt.legend()
plt.show()
```



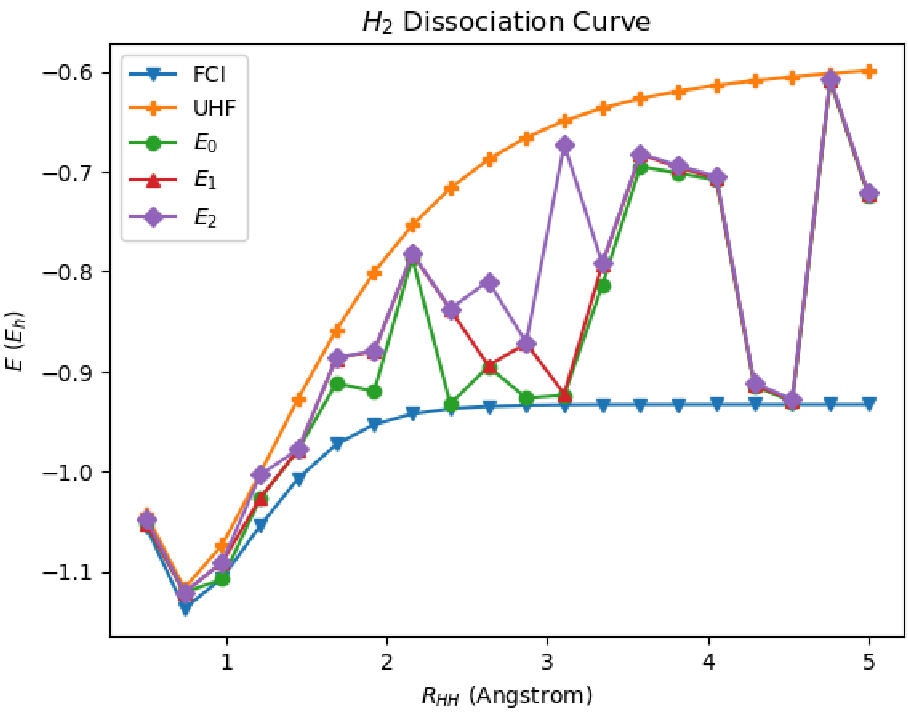
HF often provides a good initial guess near equilibrium, but it misses strong correlation at stretched bonds [9]. QPE tends to match FCI more closely at small R_{HH}, while the agreement worsens at larger R_{HH}. This suggests that the UHF state likely has greater overlap with the true ground state at shorter bond lengths. The jagged line may indicate overlap with excited states at some points, though phase discretization and branch selection may also contribute.

To address this, the code below ranks the three most probable θ values by energy, identifying the lowest as the ground-state, the second lowest as the first excited-state, and so on. Note that there is no guarantee that these correspond to E_0, E_1, and E_2, and this is only a suggestion.

In [125…

```
Es0 = []
Es1 = []
Es2 = []

for R in R_list:
    energies,FCI,UHF=Ecalc(R,k_uhf, n=8, basis="sto-3g", shots=10000, r=20, trotter_order=2)
    energies.sort()
    Es0.append(energies[0])
    Es1.append(energies[1])
    Es2.append(energies[2])


plt.figure()
plt.plot(R_list, FCIs, marker="v", label="FCI")
plt.plot(R_list, UHFs, marker="P", label="UHF")
```

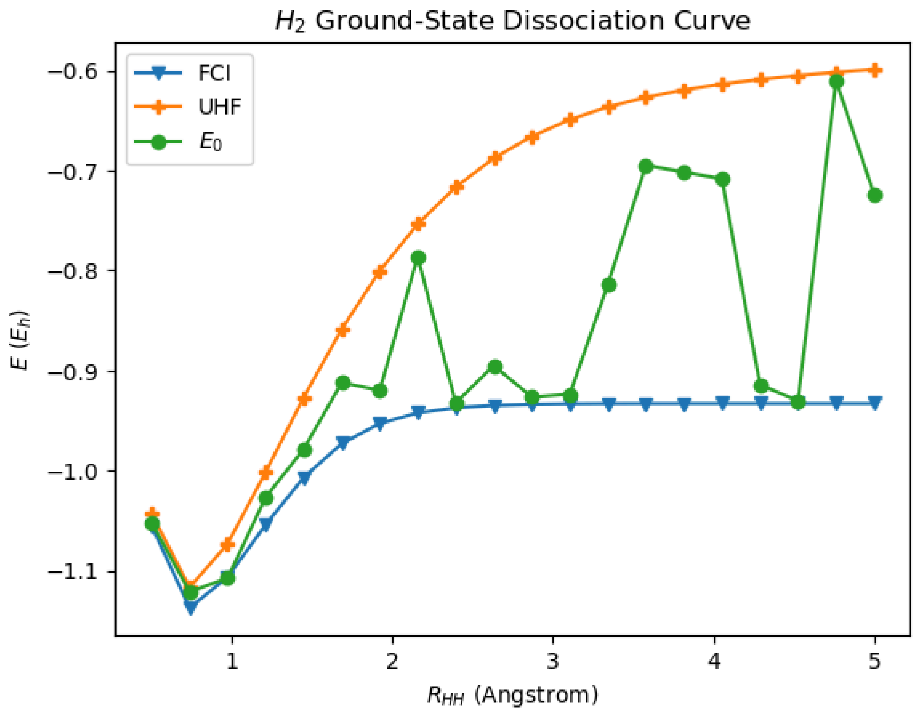
```
plt.plot(R_list, Es0, marker="o", label=r"$E_0$")
plt.plot(R_list, Es1, marker="^", label=r"$E_1$")
plt.plot(R_list, Es2, marker="D", label=r"$E_2$")

plt.xlabel(r"$R_{HH}$ (Angstrom)")
plt.ylabel(r"$E$ ($E_h$)")
plt.title("$H_2$ Dissociation Curve")
plt.legend()
plt.show()
```



Just considering E_0, taken as the lowest energy among the 3 θ values, is shown below.

In [126…

```
E0s = []

for R in R_list:
    energies,FCI,UHF=Ecalc(R,k_uhf, n=8, basis="sto-3g", shots=10000, r=20, trotter_order=2)
    energies.sort()
    E0s.append(energies[0])


plt.figure()
plt.plot(R_list, FCIs, marker="v", label="FCI")
plt.plot(R_list, UHFs, marker="P", label="UHF")
plt.plot(R_list, E0s, marker="o", label="$E_0$")
plt.xlabel(r"$R_{HH}$ (Angstrom)")
plt.ylabel(r"$E$ ($E_h$)")
plt.title("$H_2$ Ground-State Dissociation Curve")
plt.legend()
plt.show()
```

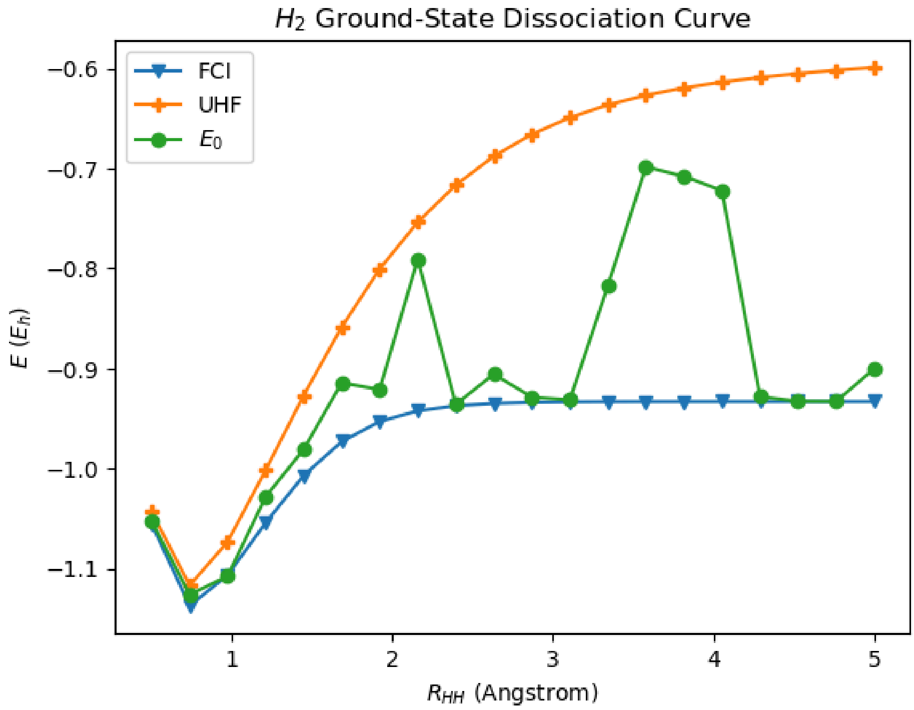


The results are slightly improved, though more θ values could also be ranked. Note that using the FCI energy as a bound is somewhat artificial, since it effectively uses knowledge of the answer when reconstructing the QPE energy. Since QPE is not variational, ranking θ values by energy can in principle produce energies below the true ground-state energy, although the FCI based lower bound used here restricts that possibility. Another issue is that the choice of initial state is crucial, and poor overlap can significantly affect the result. Even so, QPE does recover energies close to FCI at certain separations. These results show that the choice of energy bounds matters and is not straightforward.

E_0, taken as the lowest energy among the 20 θ values, is shown below.

In [127…

```
E0s = []

for R in R_list:
    energies,FCI,UHF=Ecalc(R,k_uhf, n=8, basis="sto-3g", shots=10000, r=20, trotter_order=2,num_theta=20)
    energies.sort()
    E0s.append(energies[0])


plt.figure()
plt.plot(R_list, FCIs, marker="v", label="FCI")
plt.plot(R_list, UHFs, marker="P", label="UHF")
plt.plot(R_list, E0s, marker="o", label="$E_0$")
plt.xlabel(r"$R_{HH}$ (Angstrom)")
plt.ylabel(r"$E$ ($E_h$)")
plt.title("$H_2$ Ground-State Dissociation Curve")
plt.legend()
plt.show()
```



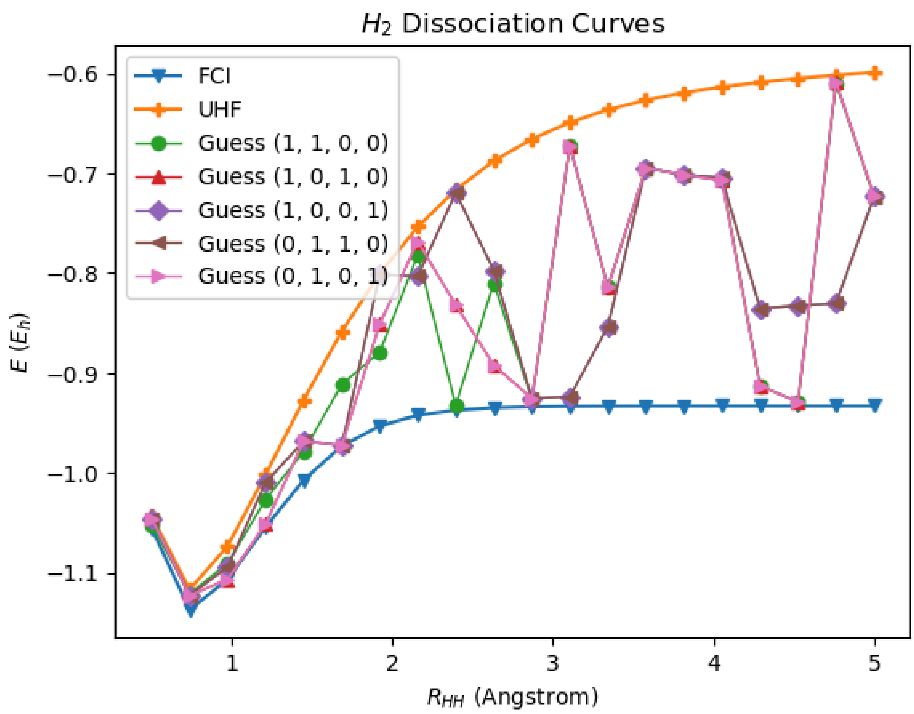
A further improvement is seen, though some points may still reflect overlap with excited states. There are many other reasons why the energies do not follow a smooth curve along the FCI line, including finite estimation precision and Trotter error.

(b)

From the ground-state analysis, it appears that reconstructed QPE energies associated with excited-state overlap can lie between the FCI and HF ground-state energies, so the same bounds are kept for the other initial state guesses.

In [128…

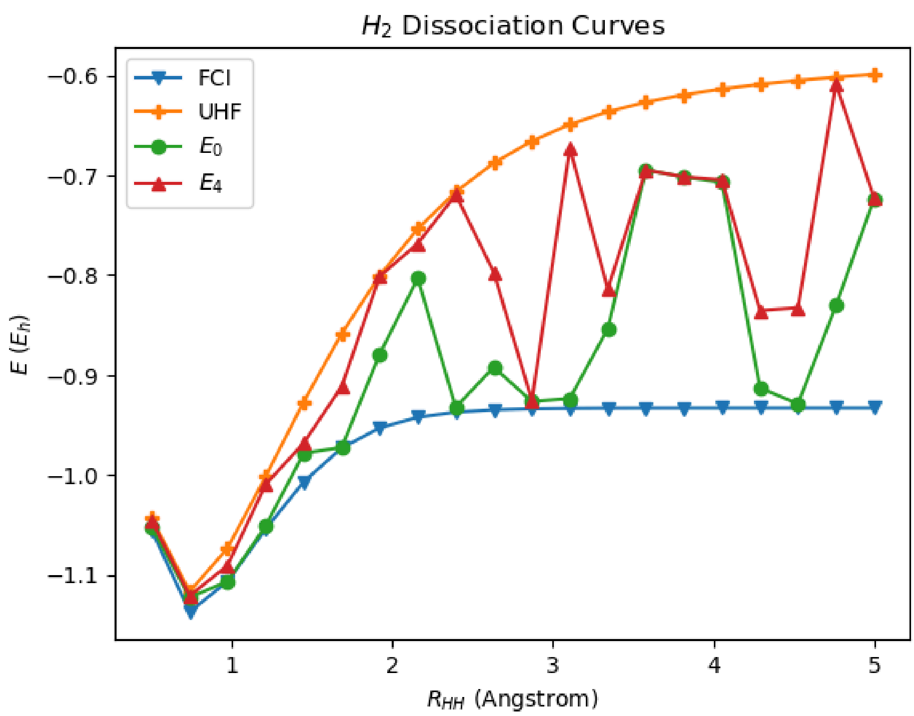
```
occ_list = [(1,1,0,0),(1,0,1,0),(1,0,0,1),(0,1,1,0),(0,1,0,1)]
markers = ["o", "^", "D", "<", ">"]
curves = {}
for occ in occ_list:
    EQPEs = []
    for R in R_list:
        energies,FCI,UHF=Ecalc(R,occ, n=8, basis="sto-3g", shots=10000, r=20, trotter_order=2)
        EQPEs.append(energies[0])

    curves[occ] = EQPEs

plt.figure()

plt.plot(R_list, FCIs, marker="v", label="FCI")
plt.plot(R_list, UHFs, marker="P", label="UHF")

for i, occ in enumerate(occ_list):
    plt.plot(R_list, curves[occ], marker=markers[i],linewidth=1, label=f"Guess {occ}")

plt.xlabel(r"$R_{HH}$ (Angstrom)")
plt.ylabel(r"$E$ ($E_h$)")
plt.title("$H_2$ Dissociation Curves")
plt.legend()
plt.show()
```



To compare the different initial-state guesses more clearly, the reconstructed QPE energies are ranked by energy at each R_{HH}. These are labeled $E_0, \ldots, E_4$ for convenience, where E_0 is the lowest and E_4 the highest. For clarity, only E_0 and E_4 are shown.

In [129…

```
E0 = []
E4 = []

for j in range(len(R_list)):
    vals = [curves[occ][j] for occ in occ_list]
    vals.sort()
    E0.append(vals[0])
    E4.append(vals[4])

plt.figure()
plt.plot(R_list, FCIs, marker="v", label="FCI")
plt.plot(R_list, UHFs, marker="P", label="UHF")
plt.plot(R_list, E0, marker="o", label=r"$E_0$")
plt.plot(R_list, E4, marker="^", label=r"$E_4$")

plt.xlabel(r"$R_{HH}$ (Angstrom)")
plt.ylabel(r"$E$ ($E_h$)")
plt.title("$H_2$ Dissociation Curves")
plt.legend()
plt.show()
```



There is agreement at some points, but overall the disagreement shows how important the choice of initial state is.

Module 4 Logbook

This lab focused on obtaining electronic energies using quantum phase estimation. Since the electronic Hamiltonian $\hat{H}$ is Hermitian but not unitary, the unitary time evolution operator $U(t) = \exp(-it\hat{H})$ can be simulated on a quantum computer. Acting on an energy eigenstate Ψ_j, this produces a phase e^{-itE_j}, so QPE can be used to estimate a phase θ with $e^{2\pi i\theta} = e^{-itE_j}$, from which the energy can be reconstructed once t and suitable energy bounds are chosen. The QPE circuit was implemented and tested by taking the most probable measured bitstrings and converting the corresponding phases into energy estimates.

A simple one qubit phase gate example was first used to validate the implementation, confirming that exact n-bit phases are recovered correctly. QPE was then applied to H_2, with the state register prepared in the unrestricted Hartree-Fock occupation state as an initial guess for the ground-state. The time evolution operator was implemented using Trotterized circuits from earlier sections, and both first order and second order decompositions were considered.

The measured phases were converted into energies by fixing t from an allowed energy interval and selecting the integer branch m that placed the reconstructed energy inside the chosen bounds. In practice, Hartree Fock and FCI energies were used to define these bounds, although the use of FCI energies makes the reconstruction somewhat artificial.

Finally, QPE dissociation curves for H_2 were compared with full configuration interaction and unrestricted Hartree Fock reference curves. The results showed that QPE can recover energies close to FCI at some bond lengths, but the agreement worsens at larger separations. Updated tests also showed that ranking several of the most probable phases can slightly improve the apparent ground state curve, while different initial occupation state guesses can lead to noticeably different reconstructed energies. Overall, the results highlight the importance of the initial state, phase precision, Trotter error, and energy bound selection. Circuit depth and simulation time were also tracked as functions of n, showing the rising cost of higher phase precision.

References

[1] Huynh, Bang. Quantum Computing for Quantum Chemistry. Laboratory manual, University of Oxford, September 2025.

[2] Young, David C. *Computational Chemistry: A Practical Guide for Applying Techniques to Real World Problems*. Wiley Interscience, 2001.

[3] *PySCF: Quickstart*. PySCF.org. https://pyscf.org/quickstart.html.

[4] Schulz, H. *Basis Sets in Quantum Chemistry*. Lecture notes, University of Rostock https://www.schulz.chemie.uni-rostock.de/storages/uni-rostock/Alle_MNF/Chemie_Schulz/Computerchemie_3/basis_sets.pdf.

[5] *OpenFermion-PySCF Demo*. Quantumlib. https://github.com/quantumlib/OpenFermion-PySCF/blob/main/examples/openfermionpyscf_demo.ipynb.

[6] *Superposition with Qiskit*. IBM Quantum Learning. https://quantum.cloud.ibm.com/learning/en/modules/quantum-mechanics/superposition-with-qiskit.

[7] *NumPy Data types*. NumPy Developers Documentation. https://numpy.org/devdocs/user/basics.types.html.

[8] Borchert, Oliver et al. *BGP Secure Routing Extension (BGP-SRx): Reference Implementation and Test Tools for Emerging BGP Security Standards*. NIST Technical Note 2060, National Institute of Standards and Technology, 2021. https://doi.org/10.6028/NIST.TN.2060.

[9] Tew, David P. *TCC Electronic Structure Module: L2 Hartree-Fock Theory*. Lecture notes, University of Oxford, 2021

[10] Tew, David P. *TCC Electronic Structure Module: L3: Configuration Interaction Theory*. Lecture notes, University of Oxford, 2021

[11] Mohr, P. J. et al. *CODATA Recommended Values of the Fundamental Physical Constants: 2022*. NIST Special Publication 961, National Institute of Standards and Technology, May 2024. https://physics.nist.gov/cuu/pdf/wall_2022.pdf

[12] *Configuration Interaction (CI) - User Guide*. PySCF.org. https://pyscf.org/user/ci.html.